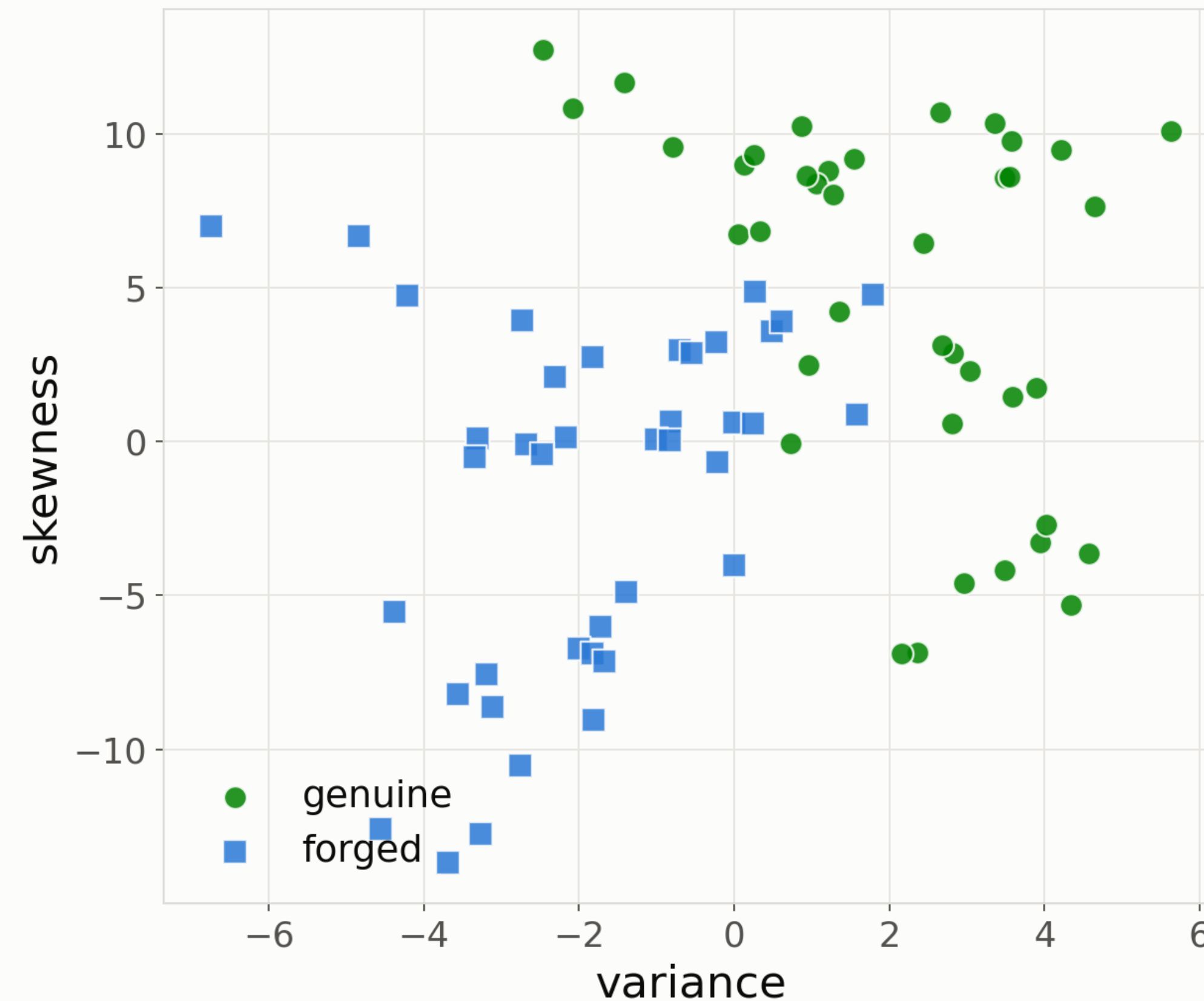


Logistic regression



10-701 Introduction to Machine Learning
Geoff Gordon

A more- interesting example



*training set
n=80 points*

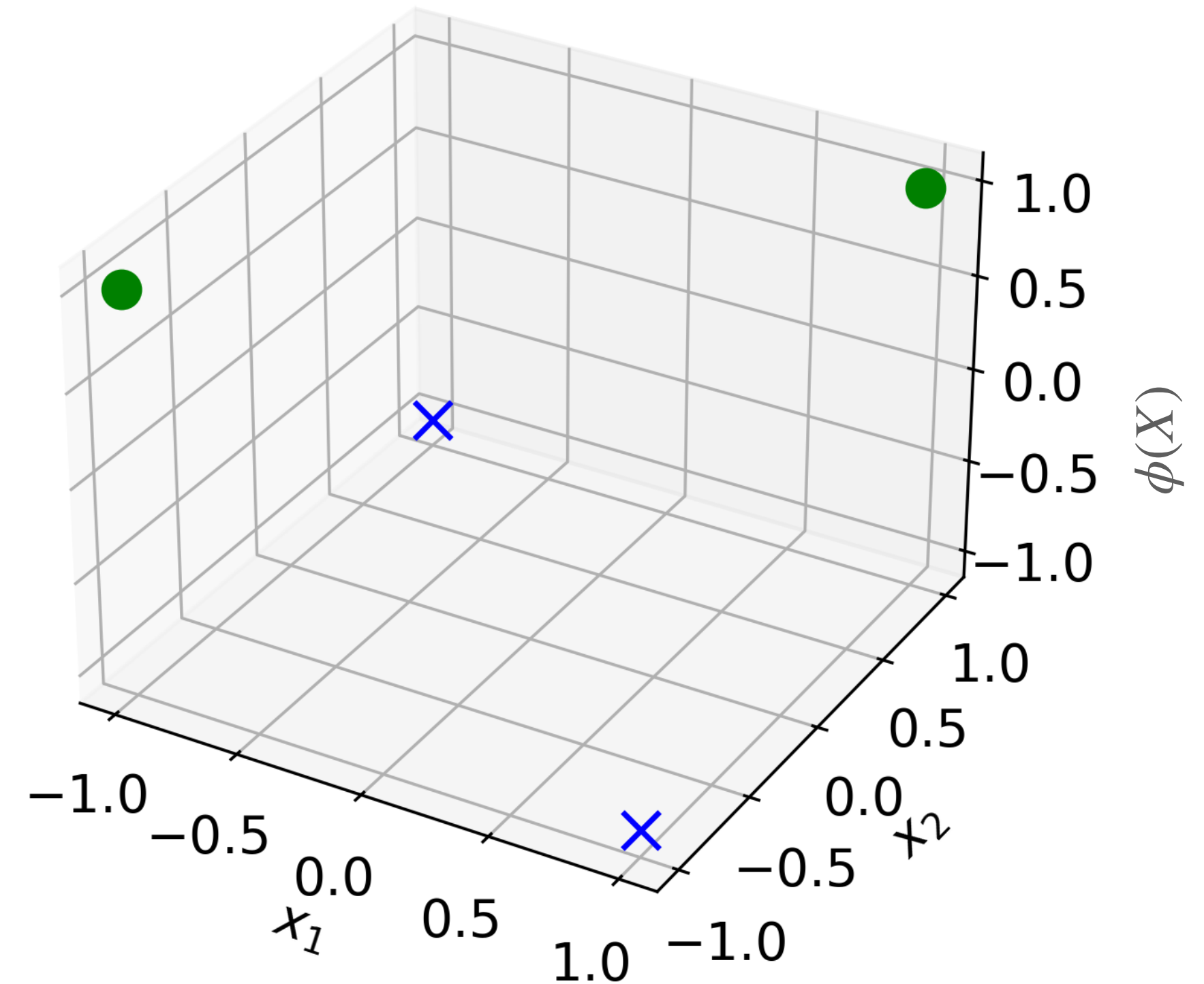
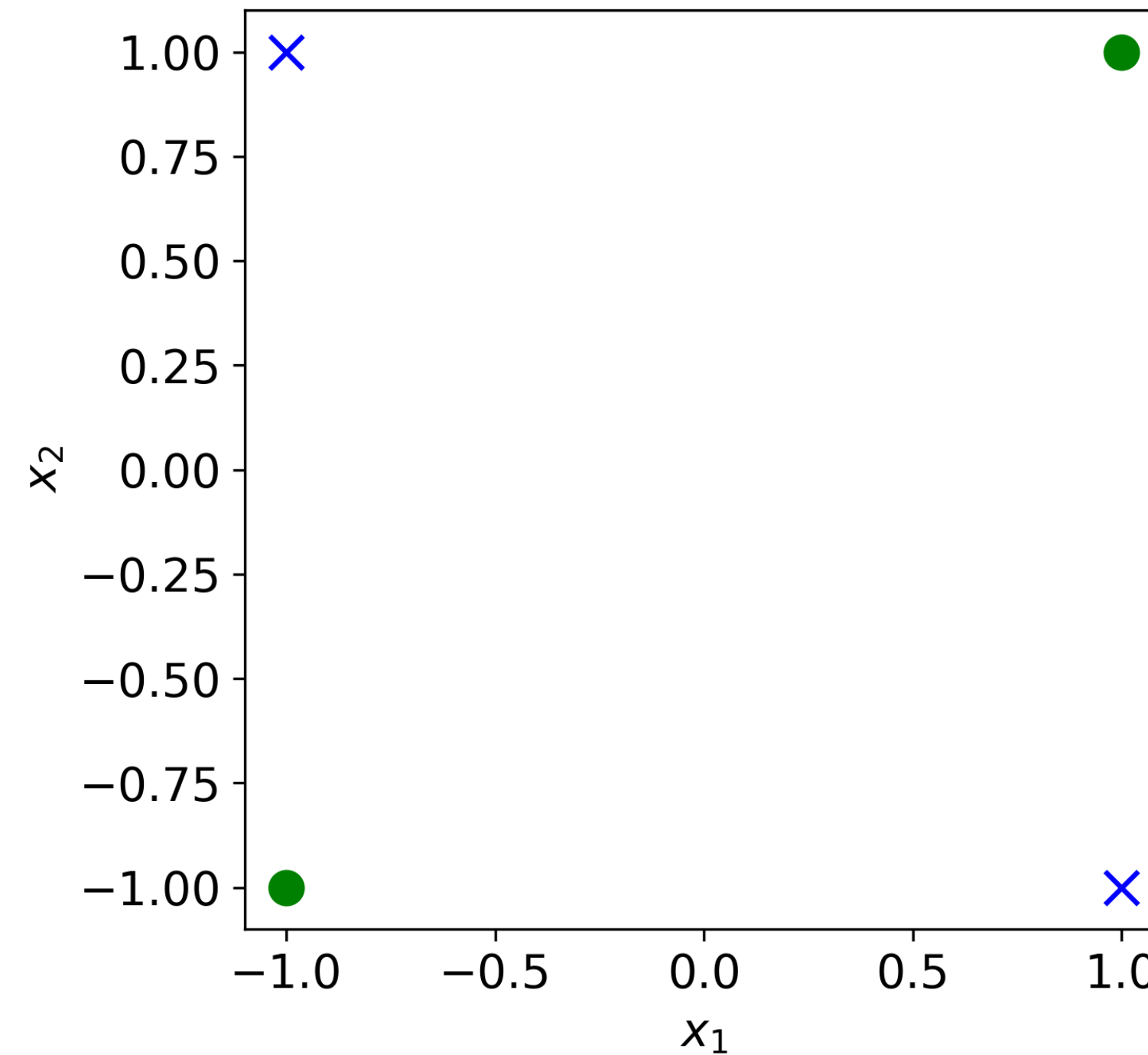
*full data has 4
features, 1372
examples*

- UCI banknote data: decide whether a banknote is genuine or forged by looking at statistics of a wavelet transform of scanned image
- Best decision boundary is not linear!

Nonlinear decision boundaries with linear models

boundary is linear in new higher-d feature space, but intersection with original feature space is nonlinear

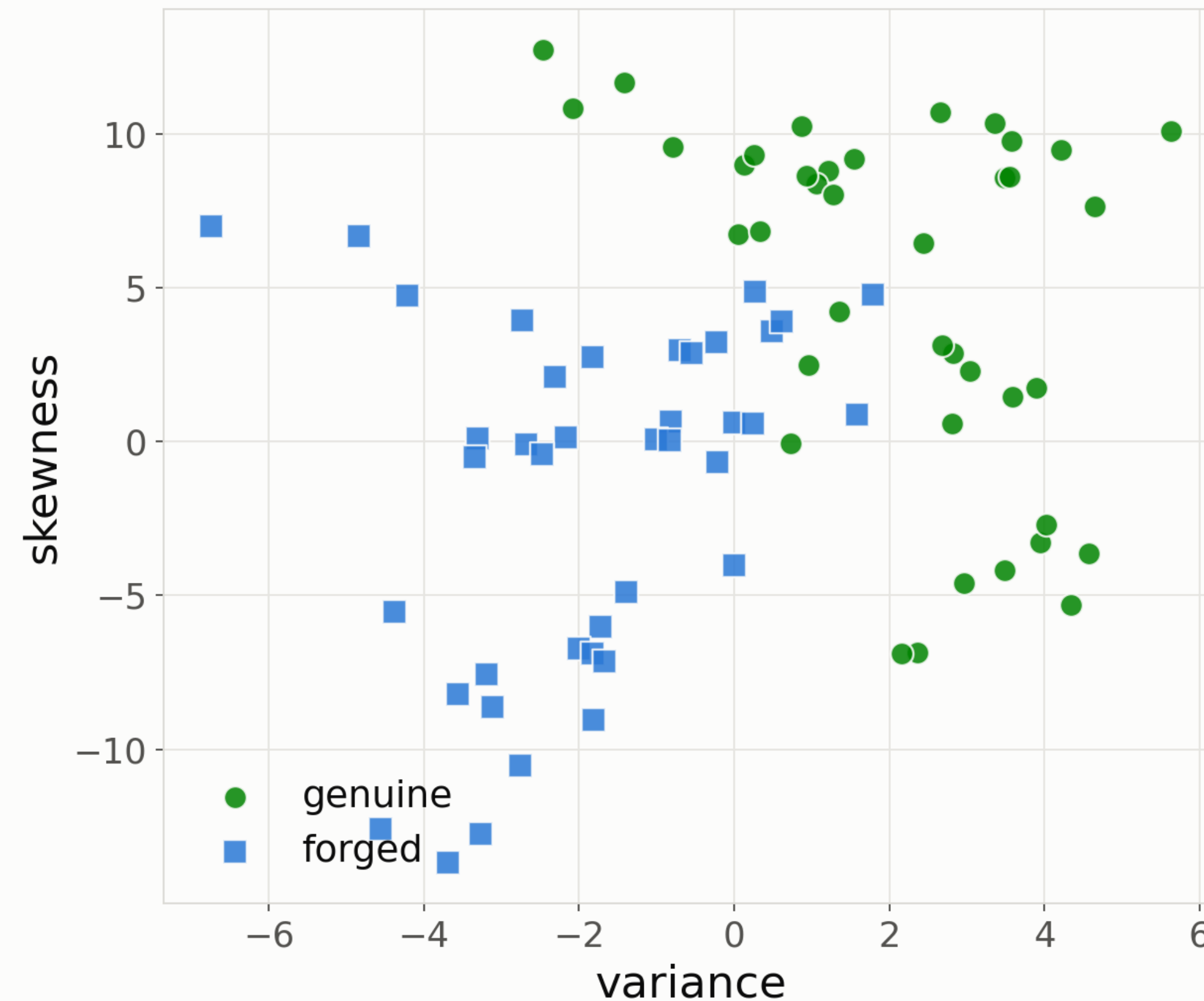
and in both views, linear in parameters



- Construct new features $\phi_1(X), \phi_2(X), \dots$: discriminant is $w_1\phi_1(X) + w_2\phi_2(X) + \dots + b$
- E.g., above 2d data is not linearly separable, but if we add a third feature X_1X_2 , we can separate it

$$\text{sgn}(0 \cdot X_1 + 0 \cdot X_2 + 1 \cdot X_1X_2 + 0)$$

How to add features to make it separable?



*training set
n=80 points*

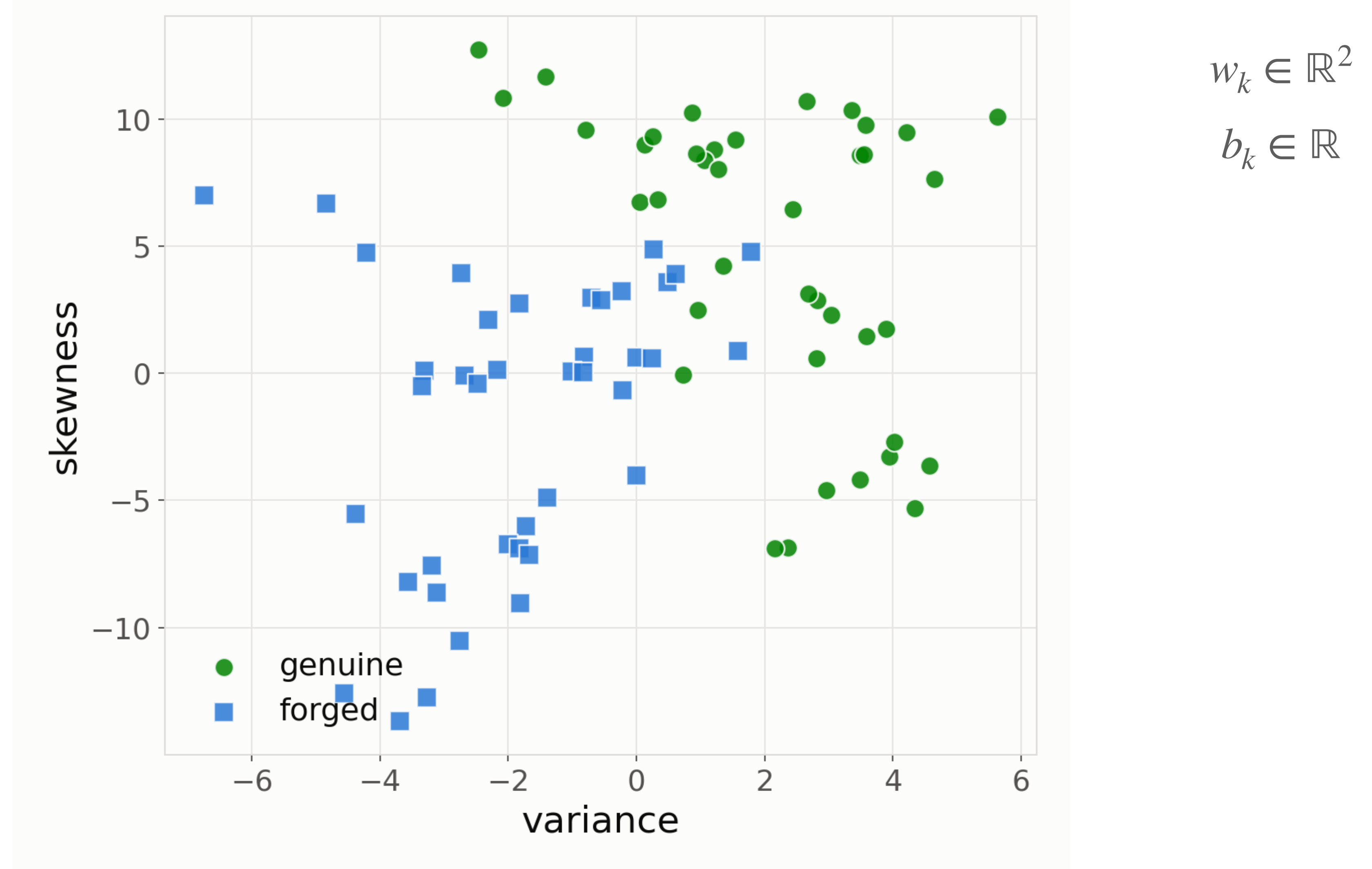
*full data has 4
features, 1372
examples*

- Problem: we don't know nearly as much about this data as we do about XOR, hard to use geometric intuition
- Need a general-purpose way to add nonlinear features

Random features

like a 2-layer neural network where we fix first-layer weights

a useful tool — some of the power of neural networks comes from random initialization

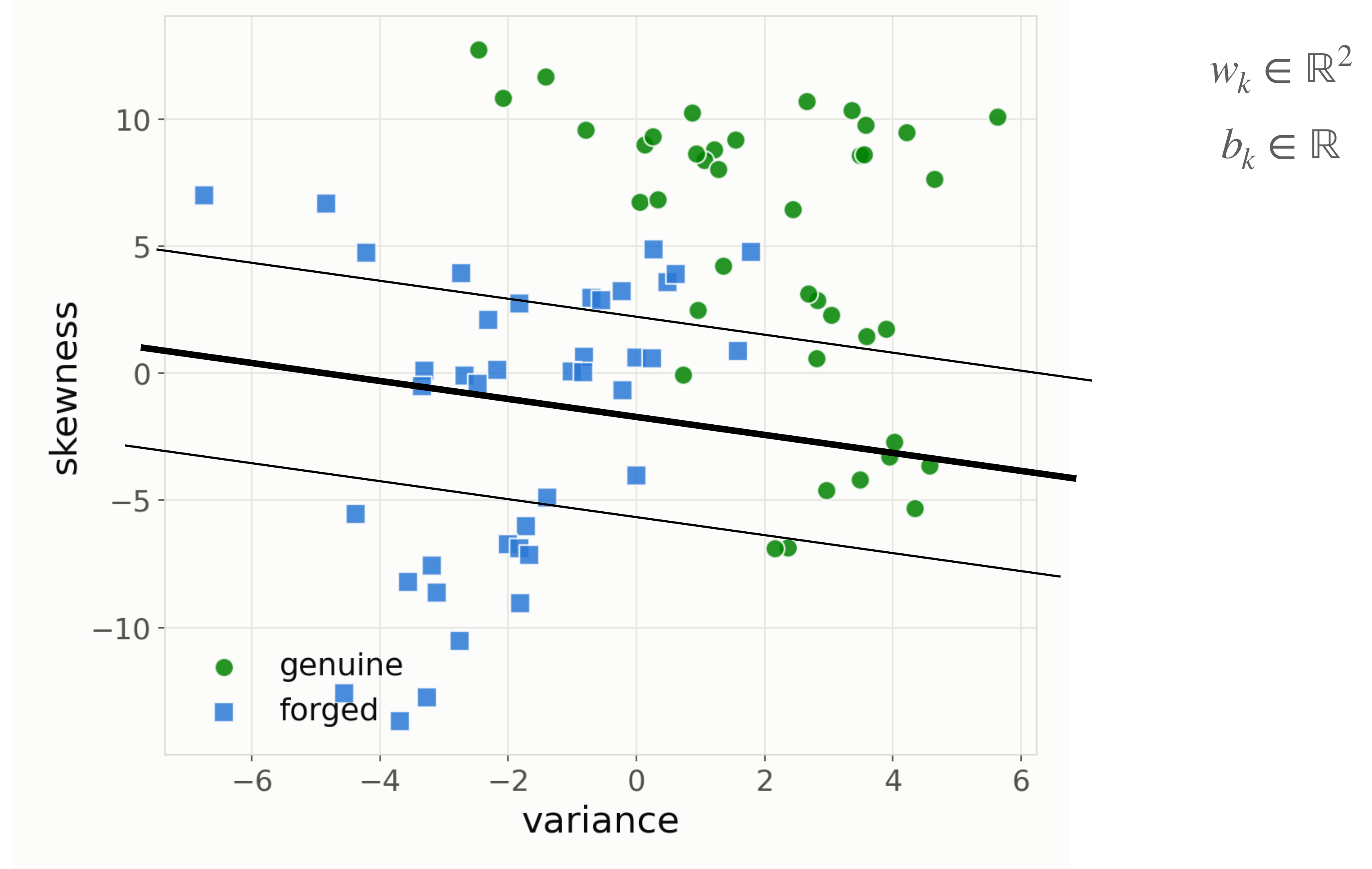


- Generate *random* features to try to make data separable
 - ▶ $\phi_k(X) = \sigma(w_k \cdot X + b_k)$, variety of positions and slopes
 - ▶ more features = more expressive decision boundary

Random features

like a 2-layer neural network where we fix first-layer weights

a useful tool — some of the power of neural networks comes from random initialization

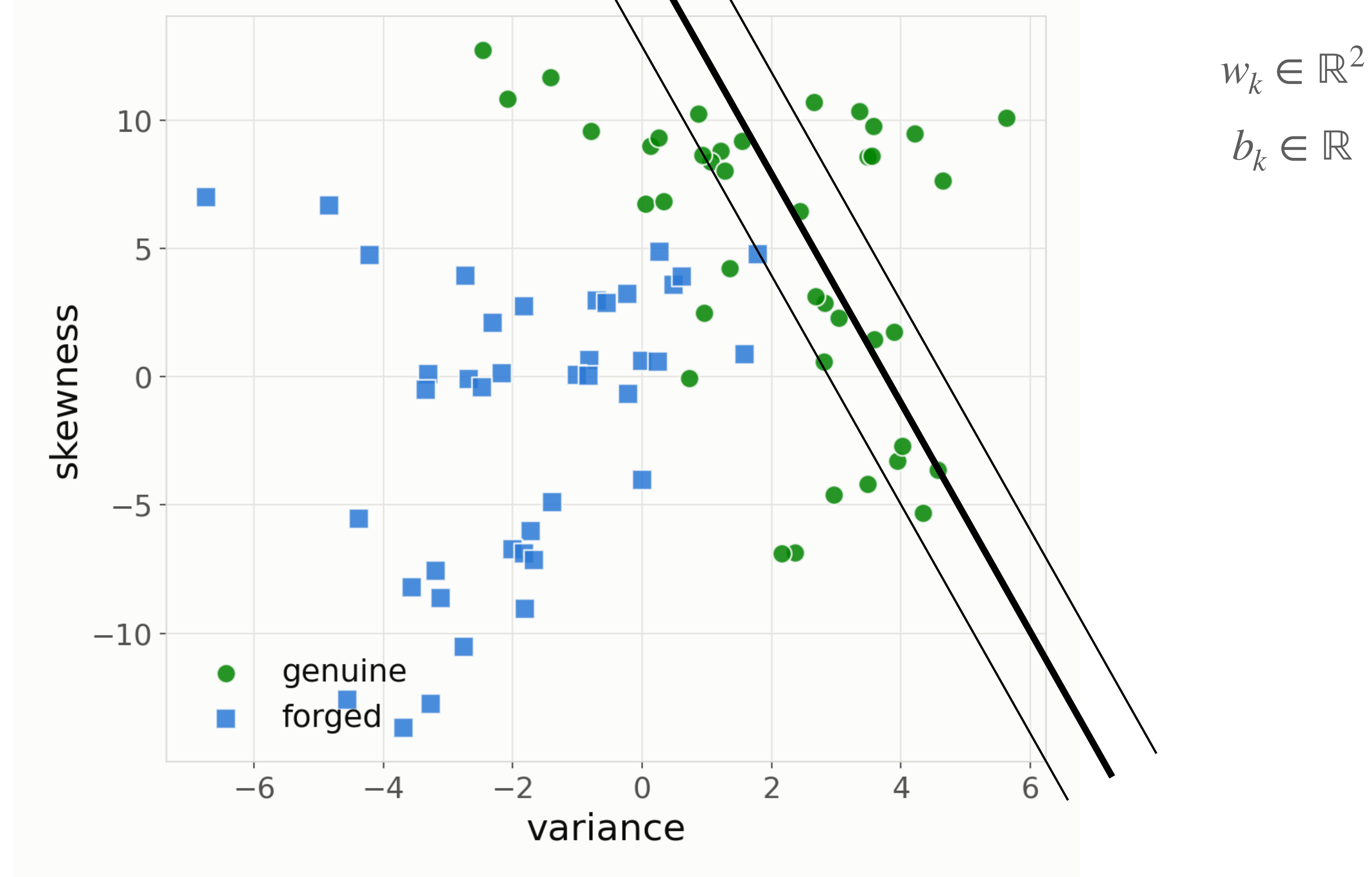


- Generate *random* features to try to make data separable
 - ▶ $\phi_k(X) = \sigma(w_k \cdot X + b_k)$, variety of positions and slopes
 - ▶ more features = more expressive decision boundary

Random features

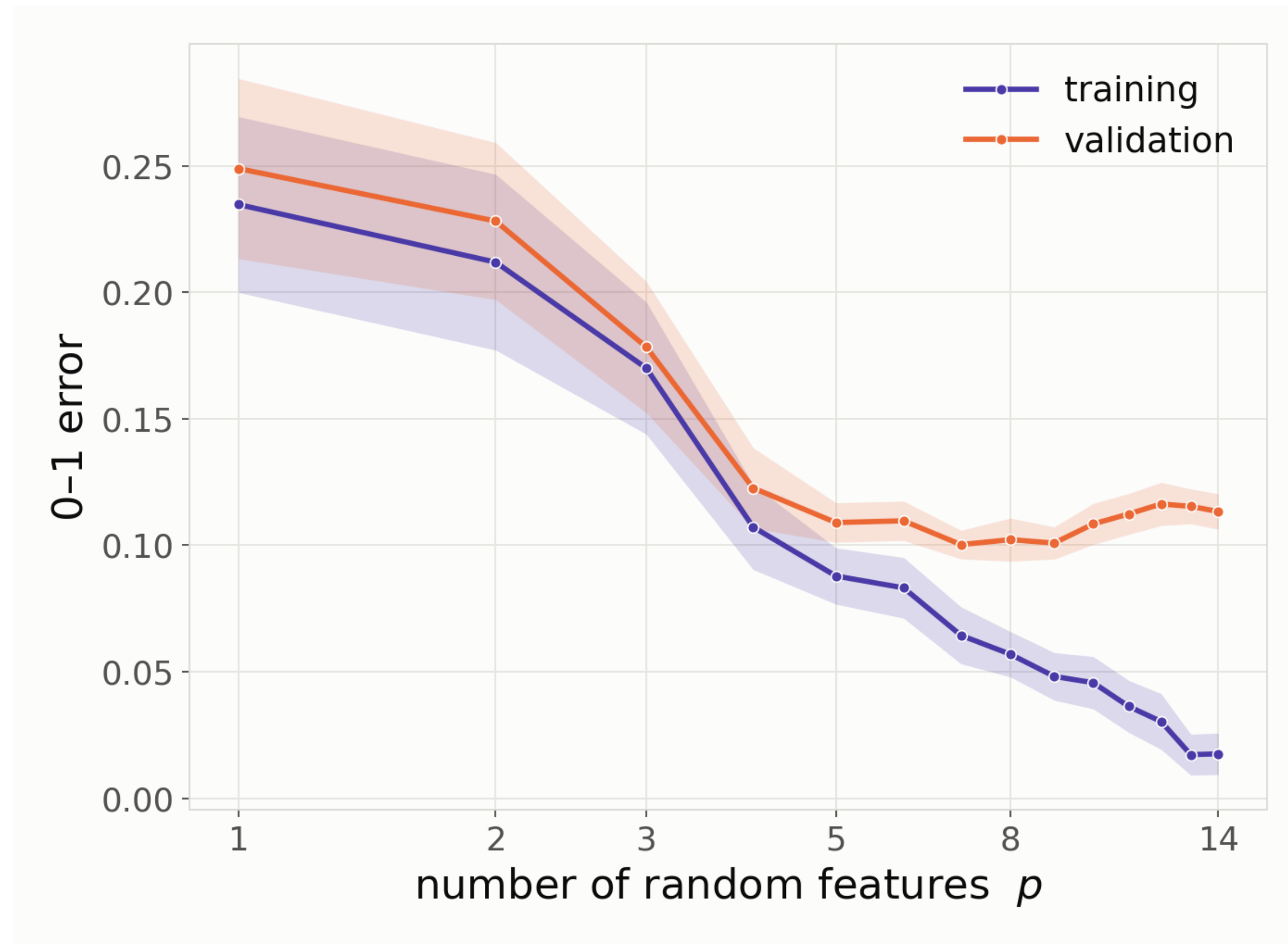
like a 2-layer neural network where we fix first-layer weights

a useful tool — some of the power of neural networks comes from random initialization



- Generate *random* features to try to make data separable
 - ▶ $\phi_k(X) = \sigma(w_k \cdot X + b_k)$, variety of positions and slopes
 - ▶ more features = more expressive decision boundary

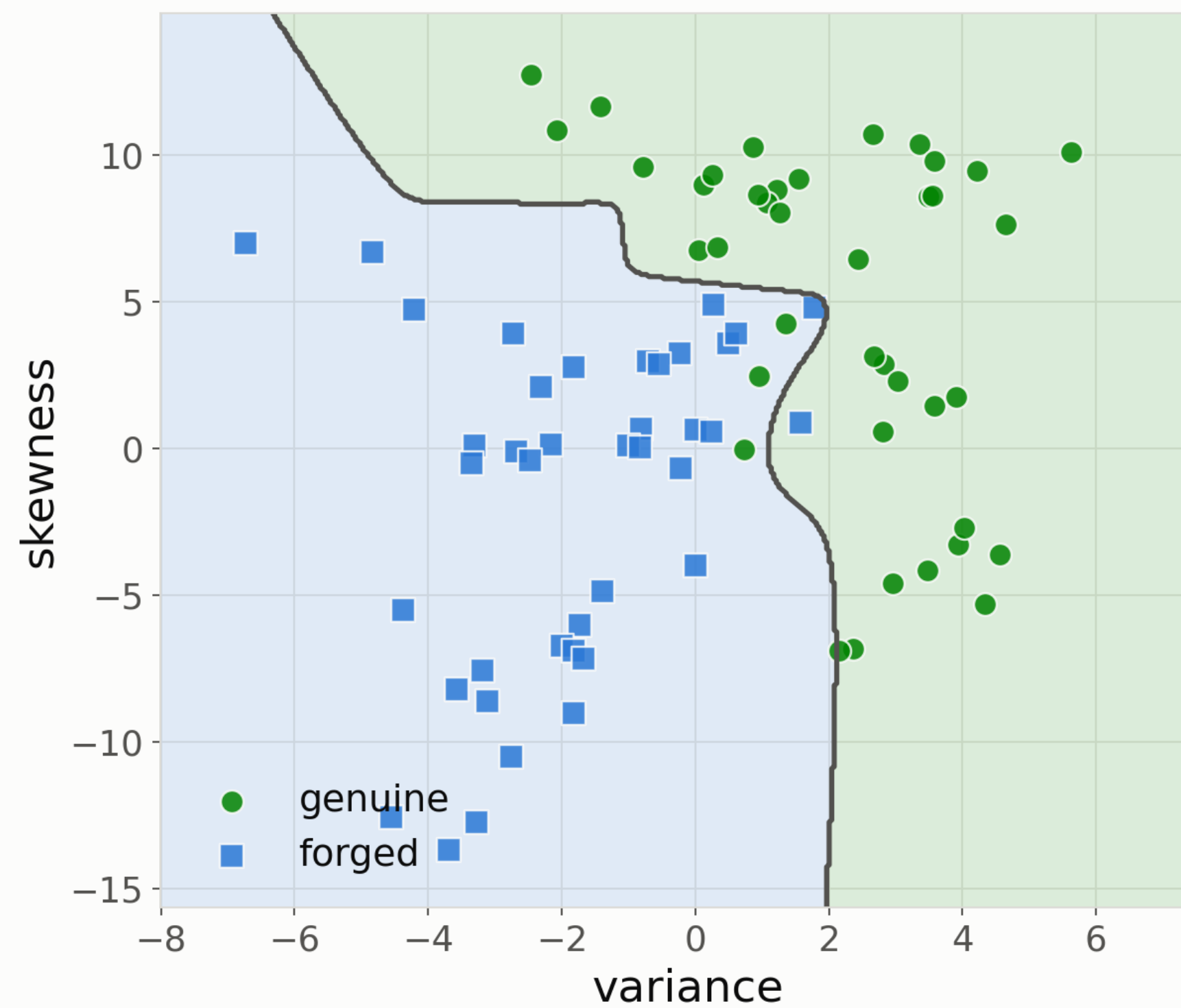
Overfitting



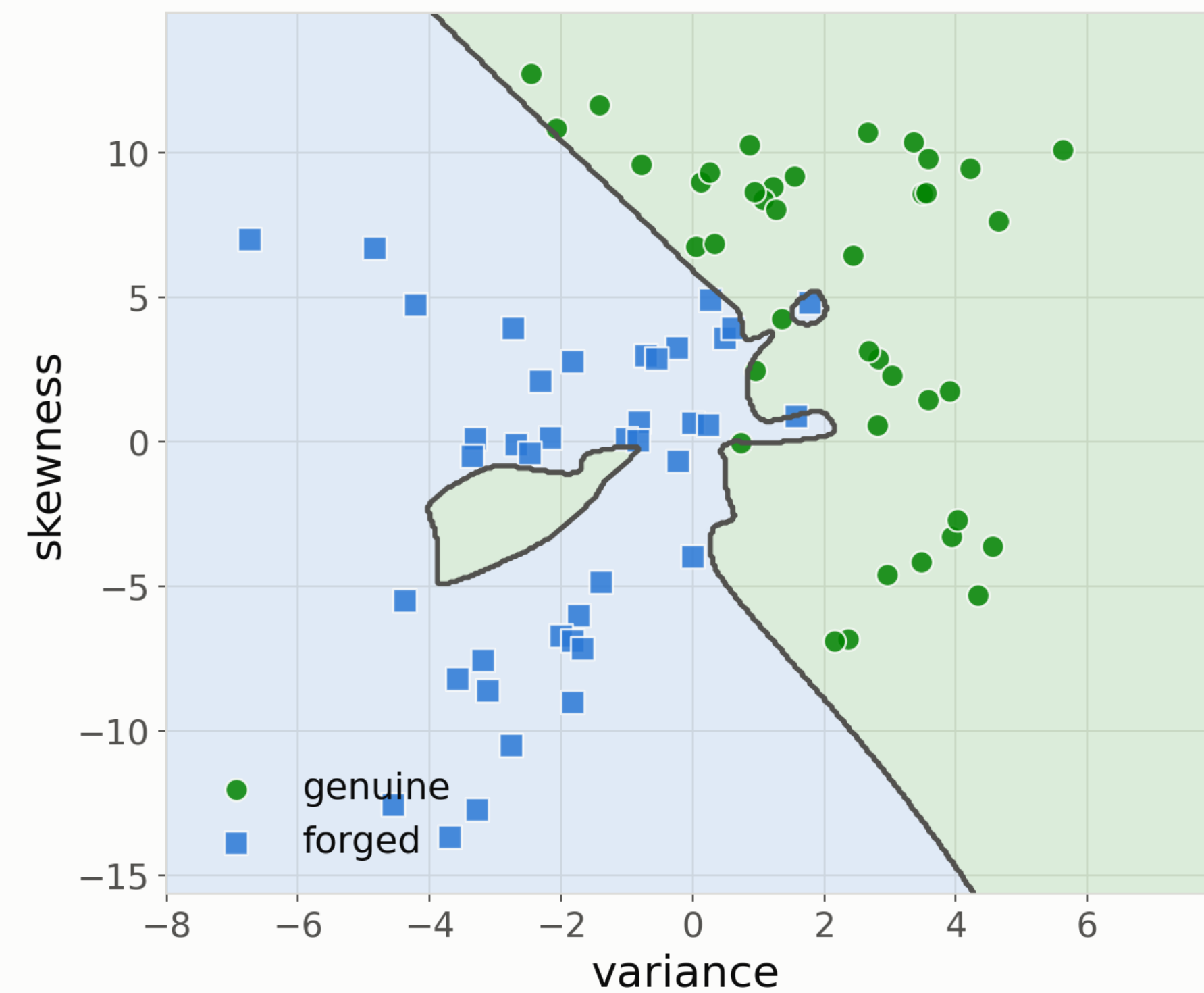
*mean $\pm 1.96\sigma$
over 40 seeds*

- As we increase the number of features, data does become more and more separable, but logistic regression overfits

Decision boundaries



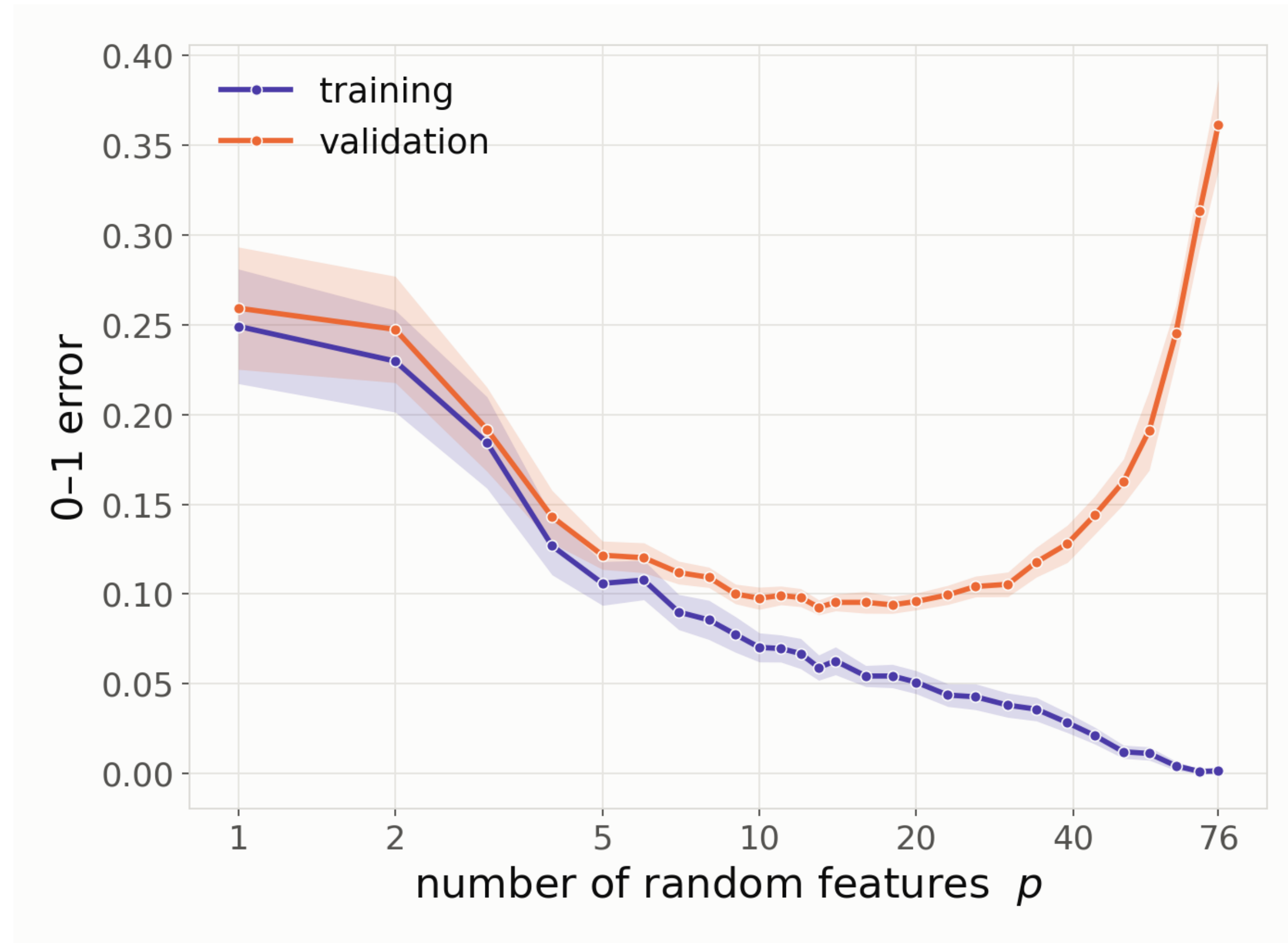
9 features



18 features

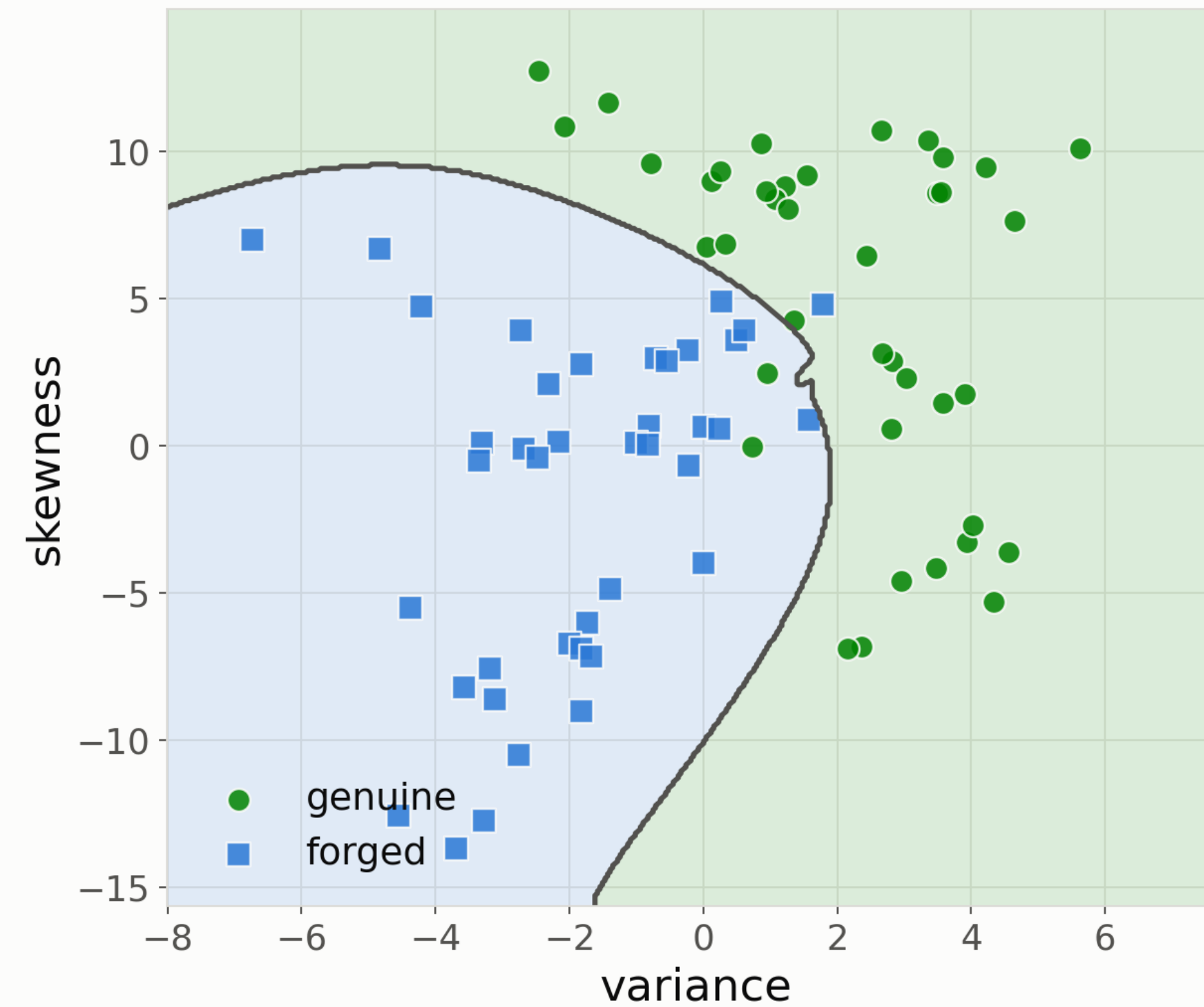
- Decision boundary gets more complex as we overfit

Overfitting

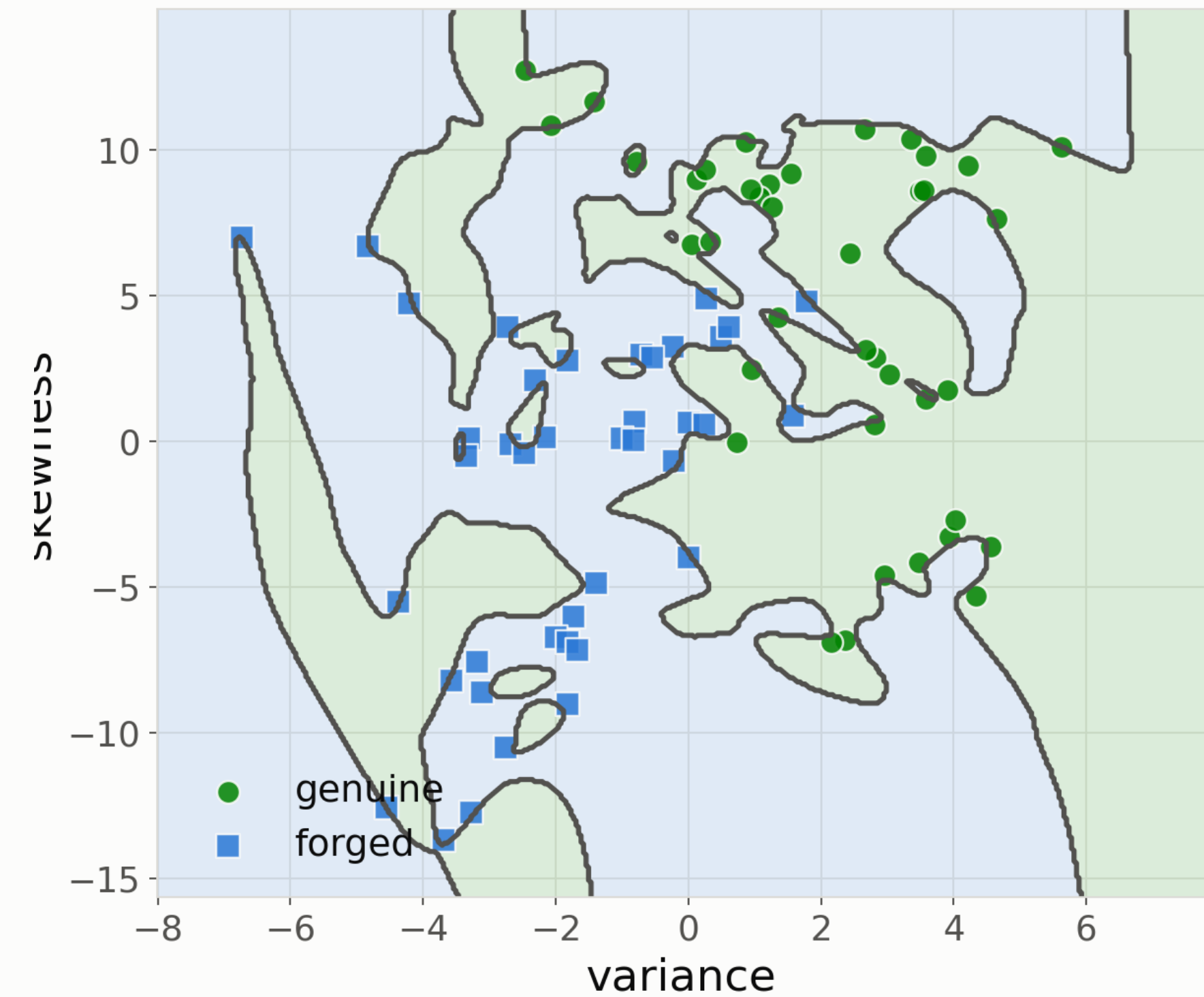


- Can run the same experiment with linear regression (minimize MSE instead of cross-entropy, targets still 0/1)
- Overfitting is worse: logistic regression benefits from its inductive bias towards high margin
- Also shifted right (regression needs more features)

Decision boundaries



13 features



76 features

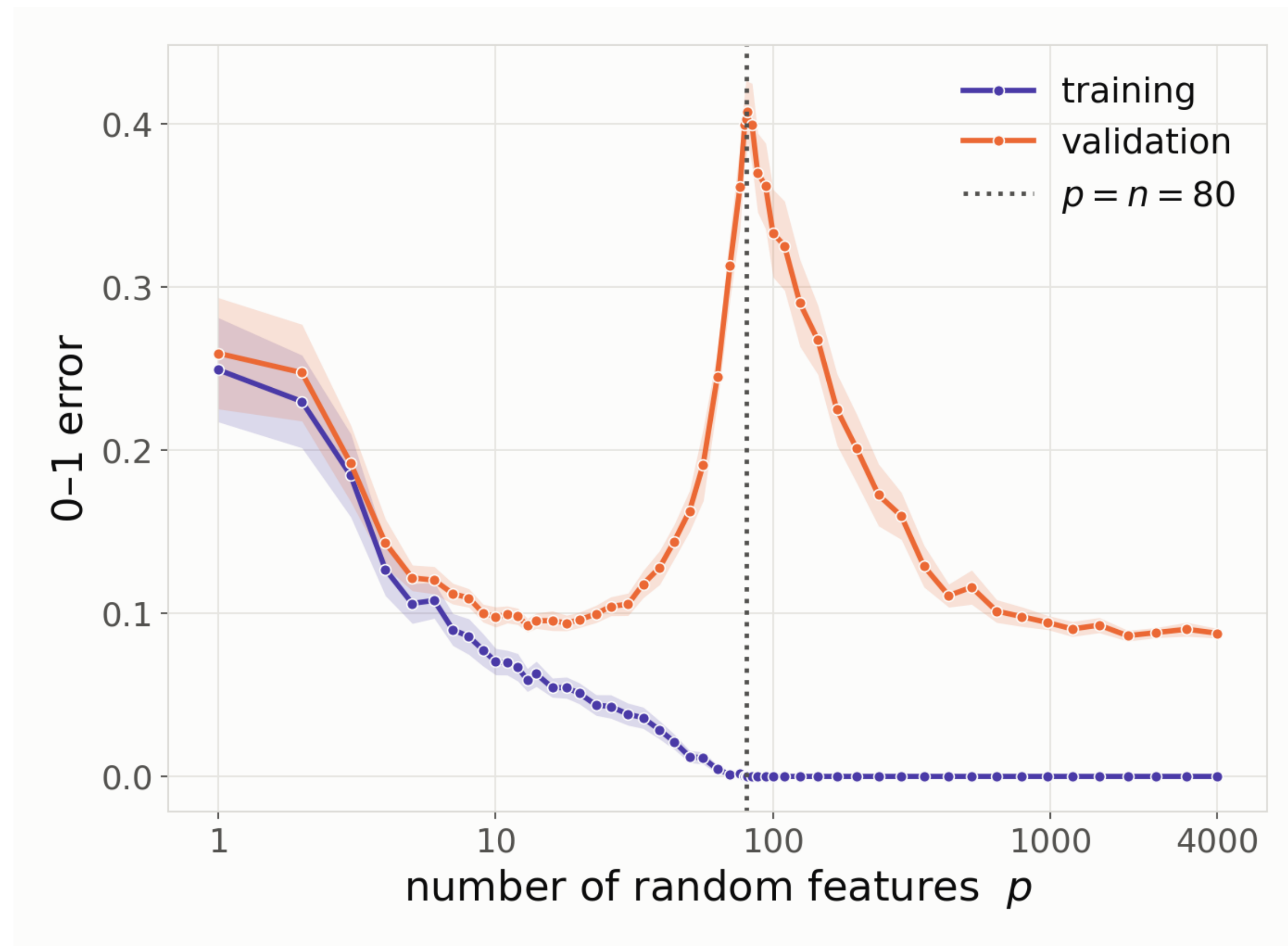
- Linear regression decision boundaries

Preview: generalization in complex models

- In the plots above, overfitting gets worse as the model class $\mathcal{H}$ gets bigger / more complex
 - ▶ and classical overfitting bounds suggest this behavior: bound allows more overfitting as $\mathcal{H}$ grows
- But more recently we've discovered cases where overfitting *goes back down* once $\mathcal{H}$ passes a threshold!
 - ▶ classical bounds become loose, some other effect takes over — we have mixed understanding of these effects
- Simple example: ***double descent*** in linear models

Double descent for linear regression

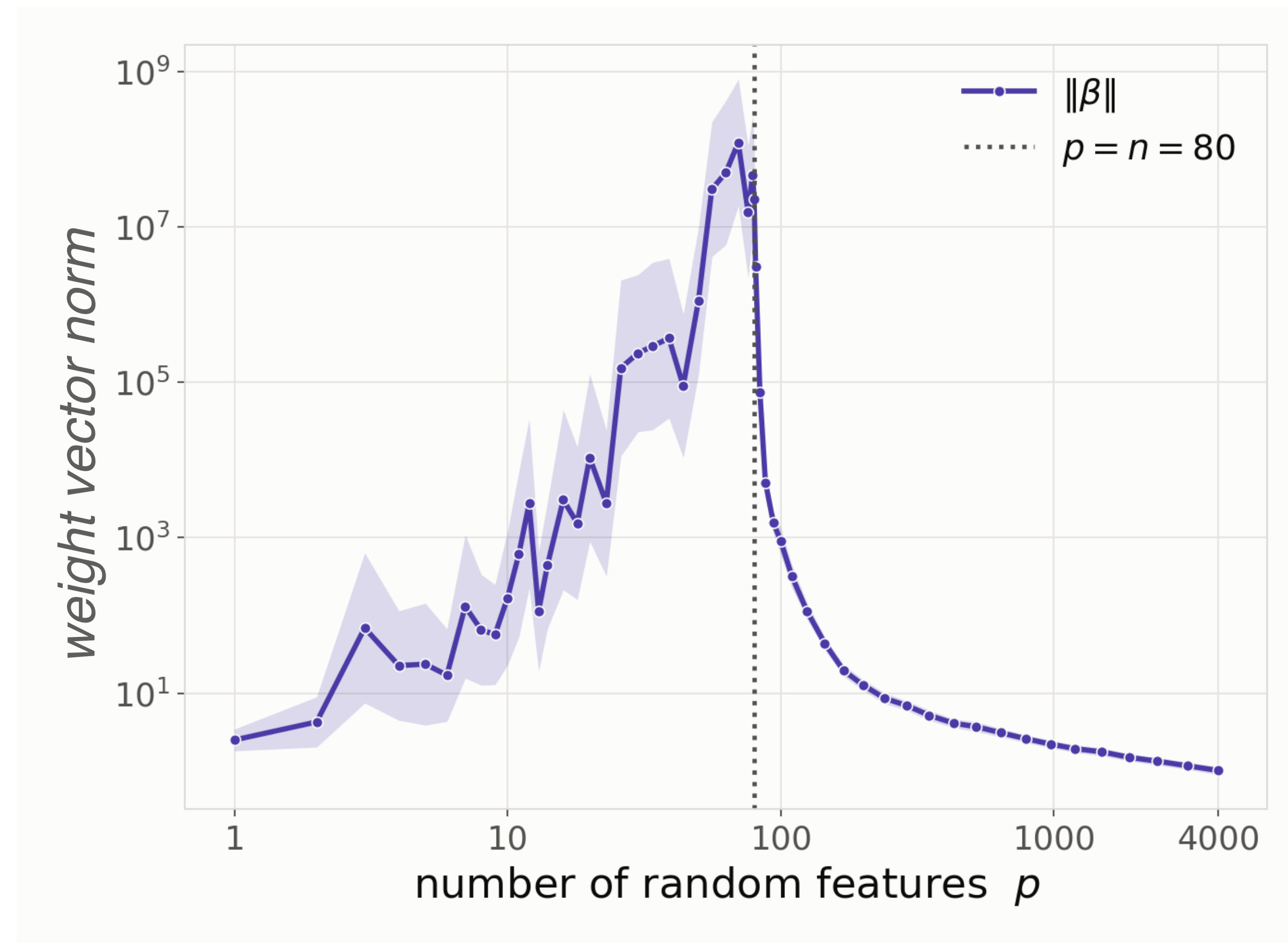
UCI banknote data



- Validation error peaks at # features = # examples, decreases after (note semilog plot)
 - ▶ gets even slightly lower than for best small model
- Training error = hard 0 from 80 features on

Double descent for linear regression

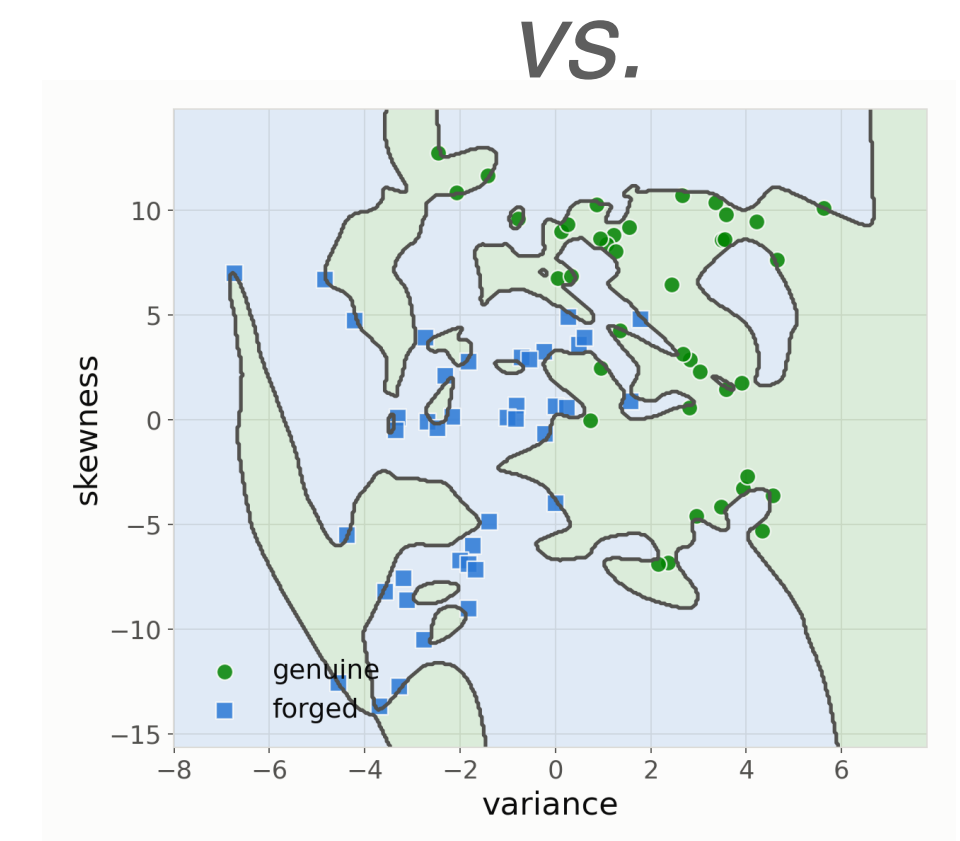
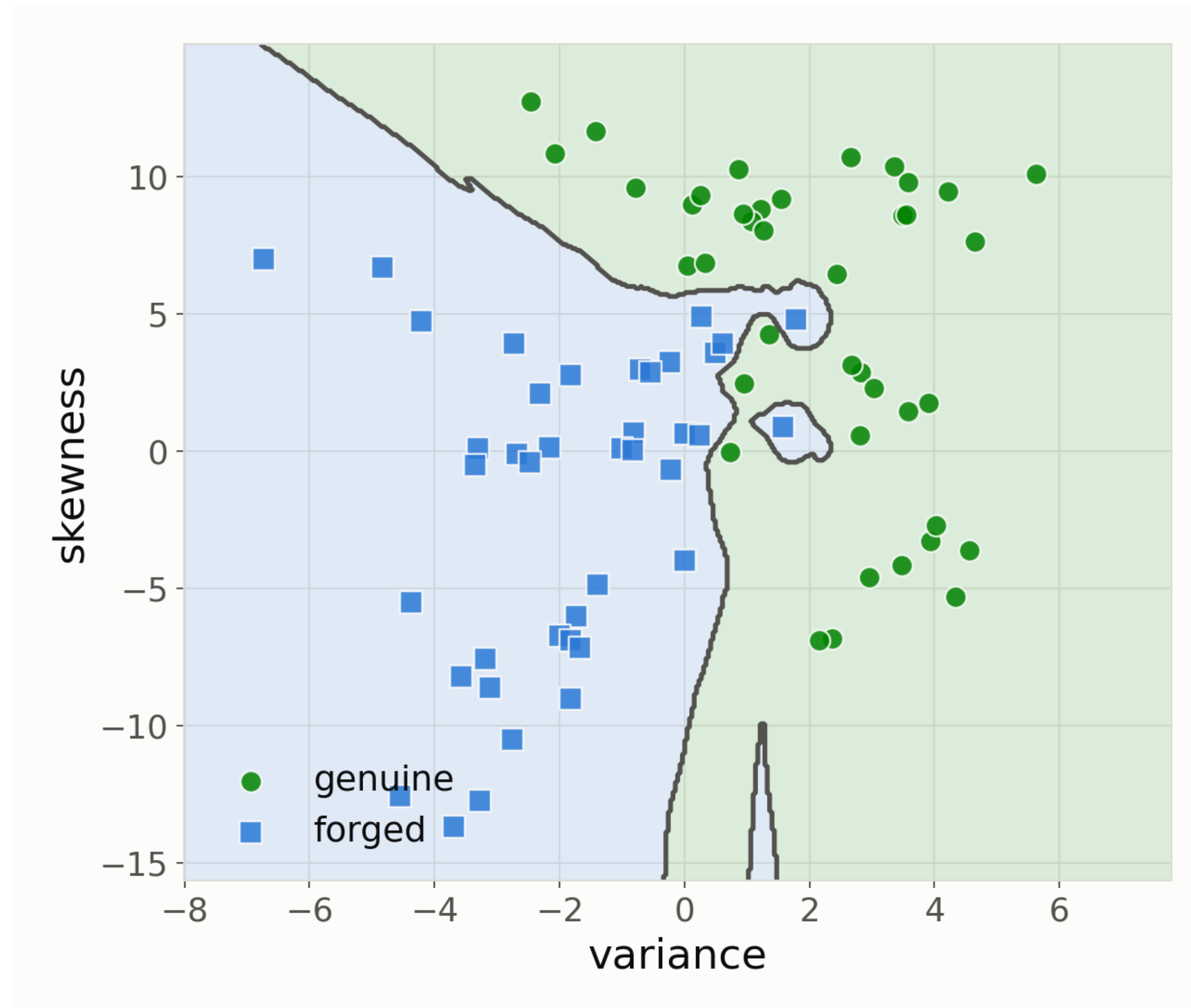
UCI banknote data



- Hint to why we get pattern on previous slide: solution norm spikes at 80 features (note log-log plot)
 - ▶ norm at 4000 features is actually lower than for best small-model solution

Double descent for linear regression

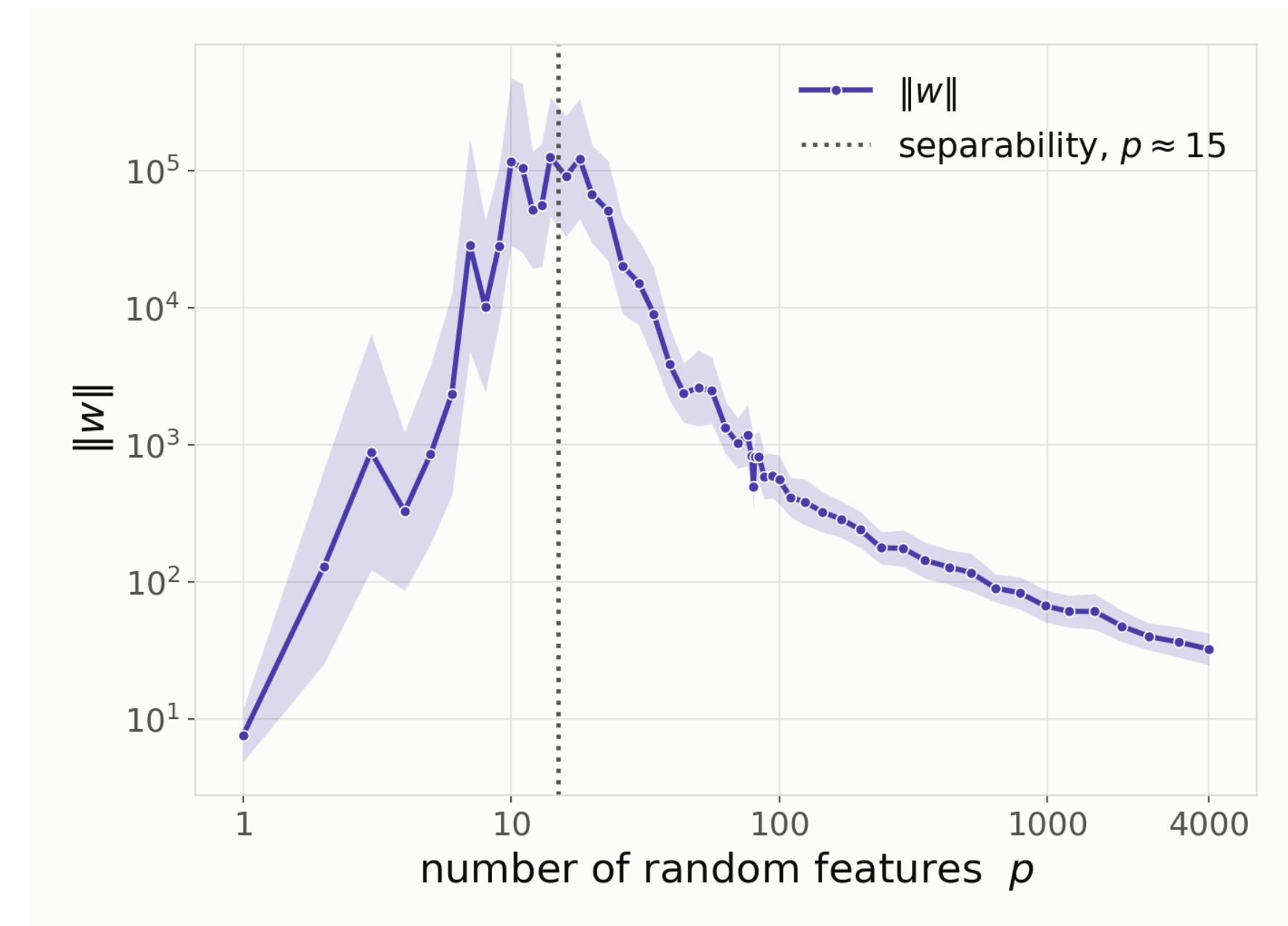
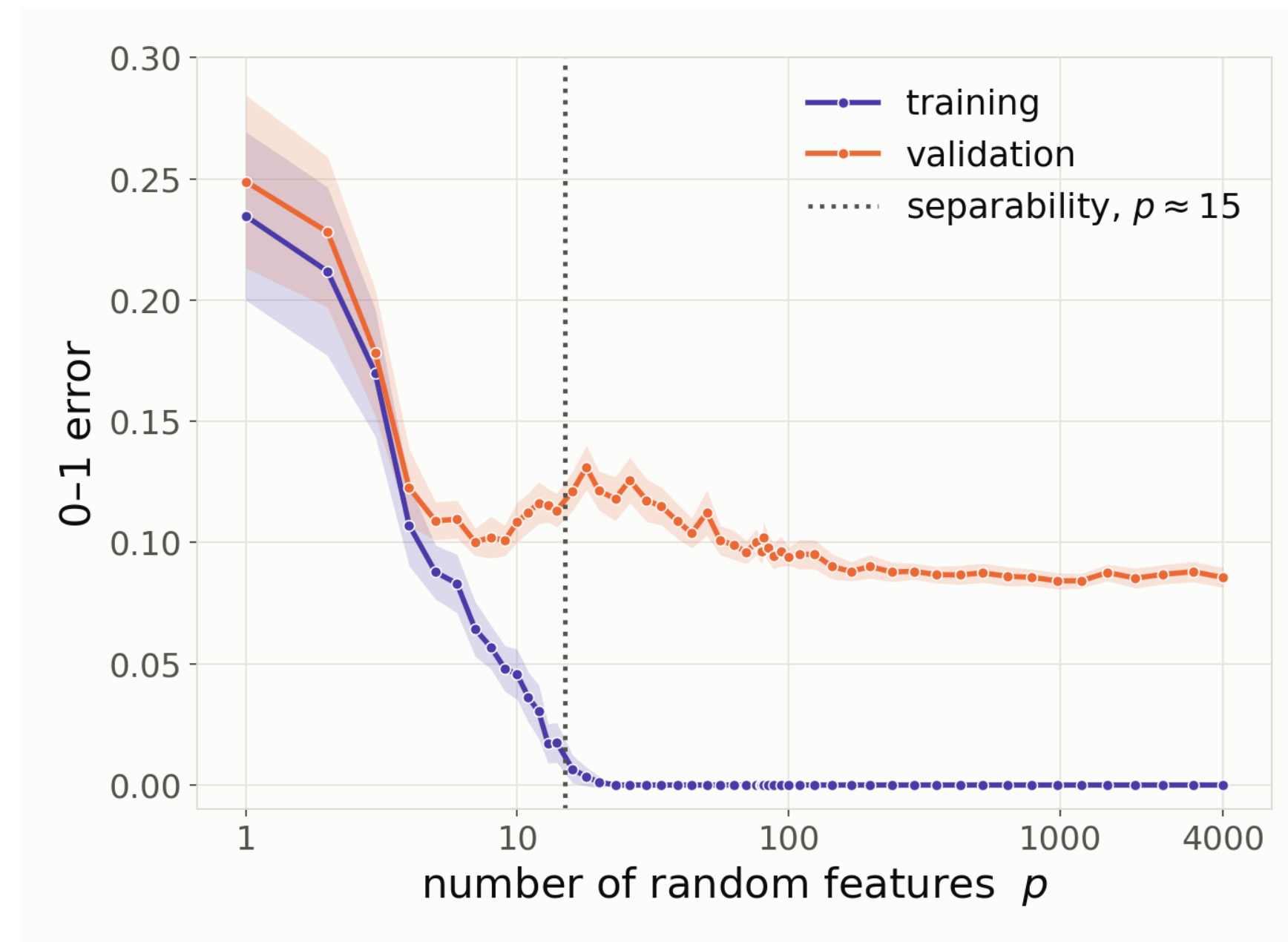
UCI banknote data



- Decision boundary gets simpler again too

Double descent for logistic regression

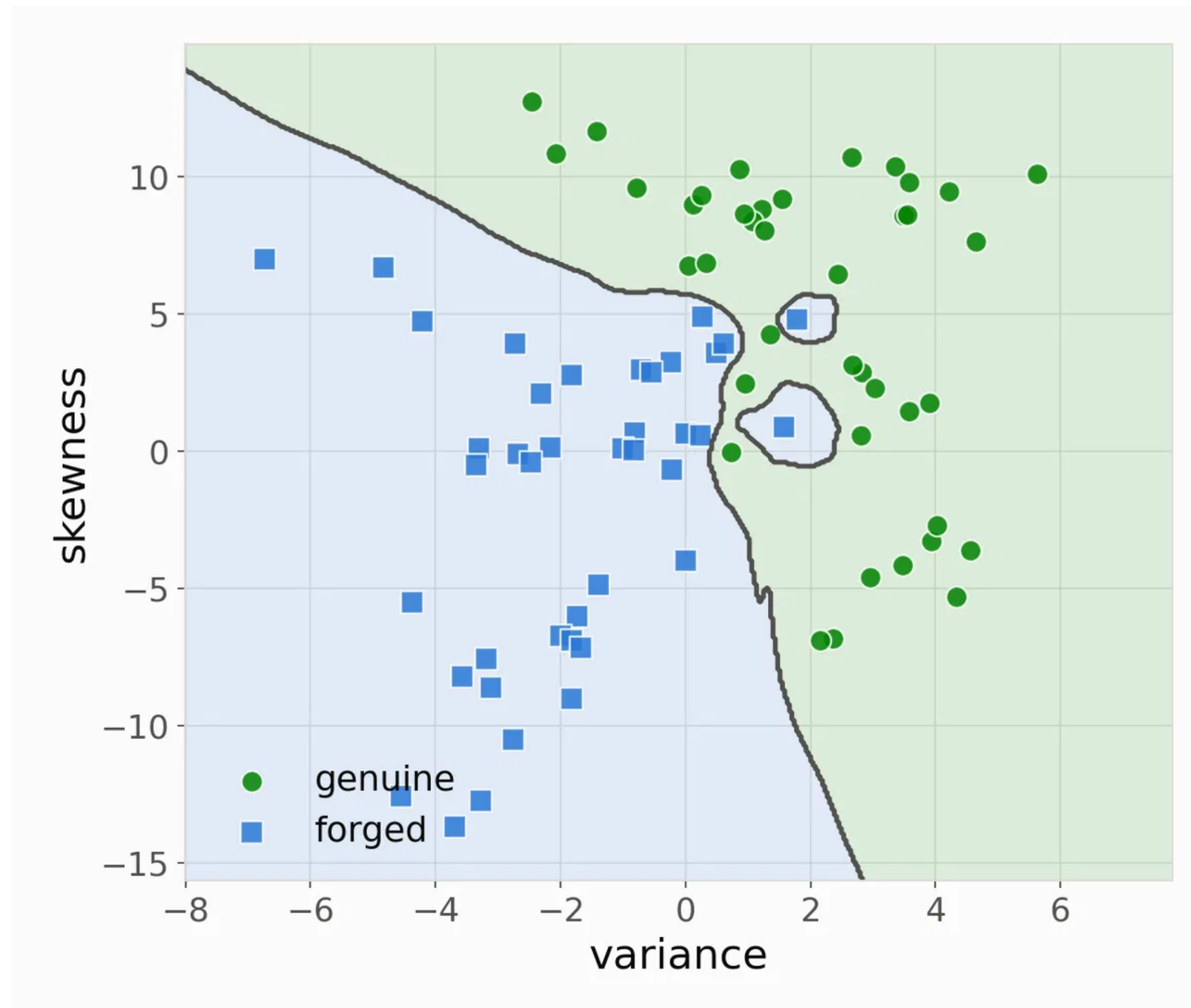
UCI banknote data



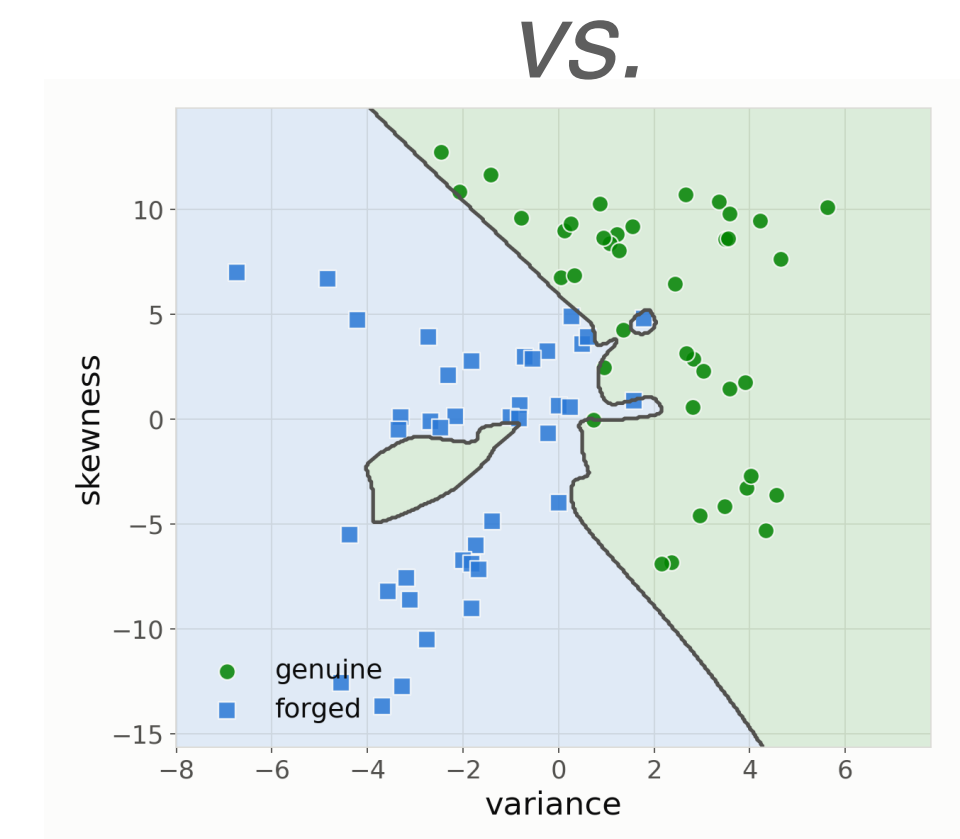
- Similar effect for logistic regression, but
 - ▶ less pronounced peak
 - ▶ shifts left (fewer features)
 - ▶ due to difference between interpolation and separation

Double descent for logistic regression

UCI banknote data



- Decision boundary



Learning objectives— logistic regression

- What is the log-likelihood of a Bernoulli distribution? A categorical distribution?
- How do they relate to the sigmoid, softmax, and cross-entropy functions?
- What is a logit or log-odds? What is a logistic regression model? What is its multi-class extension?
- How can we use maximum (conditional) likelihood and SGD to fit logistic regression?

***Learning
objectives—
Bayes,
naive Bayes,
discriminative,
constructed
features***

- What is a Bayes optimal classifier? How does it relate to naive Bayes? To logistic regression?
- What are the naive Bayes classifiers for Gaussian or categorical features?
- How can we use a classifier to learn a distribution?
- How can we learn a nonlinear decision boundary using constructed features?
- How do extra features affect overfitting?
- What is double descent?

Deep nets



10-701 Introduction to Machine Learning
Geoff Gordon

Model = learnable program

Logistic regression

$z \leftarrow \sum_i w_i x_i$
return $\sigma(z)$

Perceptron

$z \leftarrow \sum_i w_i x_i$
if $z \geq 0$:
 return T
else:
 return F

Decision tree

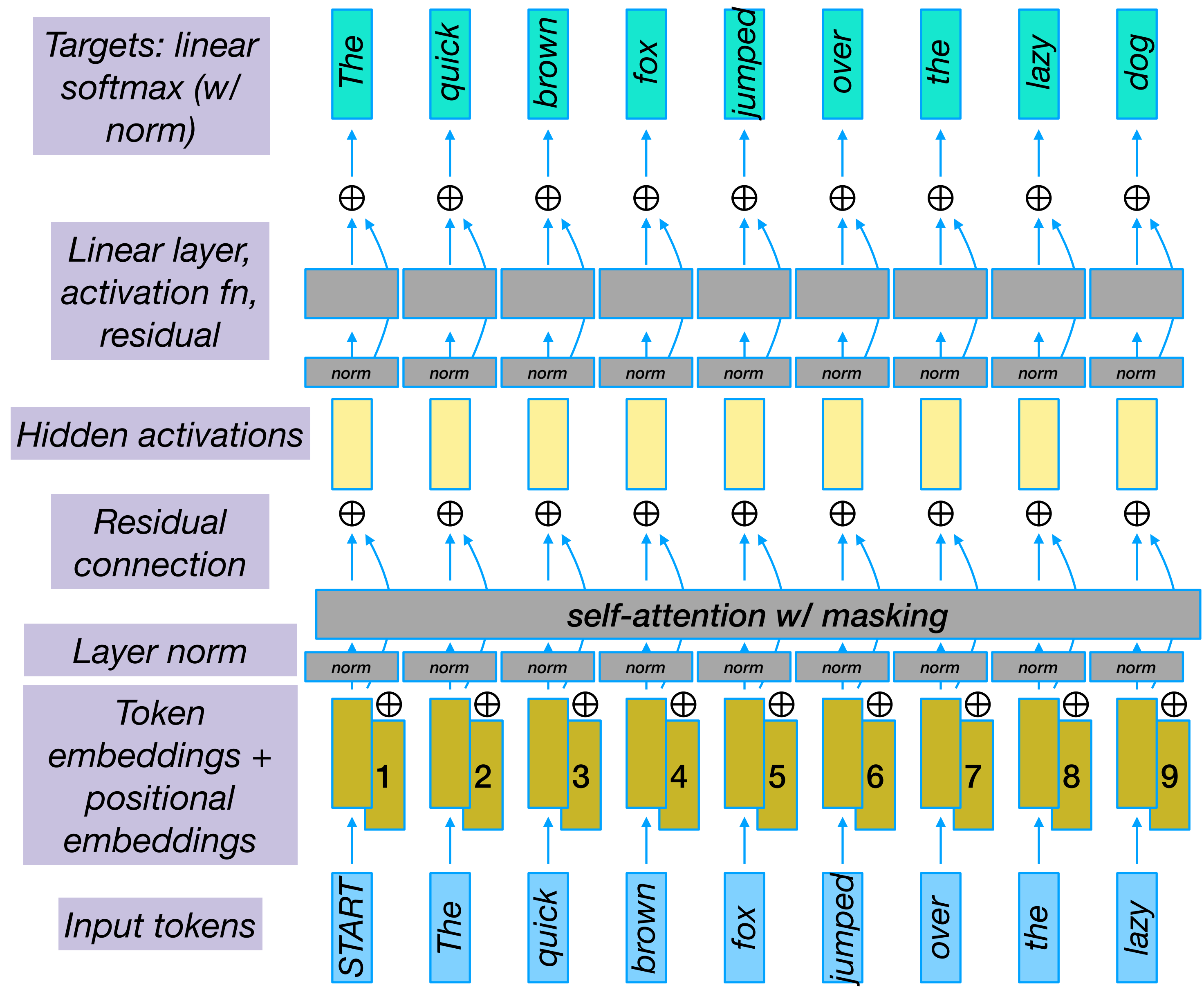
if $x_1 \leq 3$:
 if $x_2 \leq 5$:
 return T
 else:
 return F
else:
 return T

- Can think of a model as a *decision function (program)* that has *learnable choices*
 - ▶ weight vector w in perceptron or logistic regression
 - ▶ split axes and thresholds in decision tree

Neural networks

- Decision functions composed from blocks of
 - ▶ trainable linear functions
 - ▶ fixed (not trainable) nonlinear functions, often chosen from a small list like $\sigma(z)$, $z/\|z\|$, $[z]_+$, $z_1 \circ z_2$
- Often:
 - ▶ entire function is continuously differentiable at least once wrt parameters (sometimes w/ sparse breakpoints)
 - ▶ trained by SGD or variations
 - ▶ with depth $\gg 1$
- Includes linear & logistic regression, perceptron as special cases

Neural networks



Deep neural networks

notation:

$$\max(0, z) = [z]_+$$

*is **ReLU** or **positive part**,
one of the most common
nonlinearities / activation
functions for deep nets*

- Defn: ***depth*** = minimum number of sequential steps in a computation, given arbitrarily many parallel processors
- In contrast to ***width*** = max number of processors used, or ***work*** = total number of processor-steps
- E.g.,
 - ▶ $\max(0, 1 + 3 \cdot \max(0, 4x))$

What is depth?

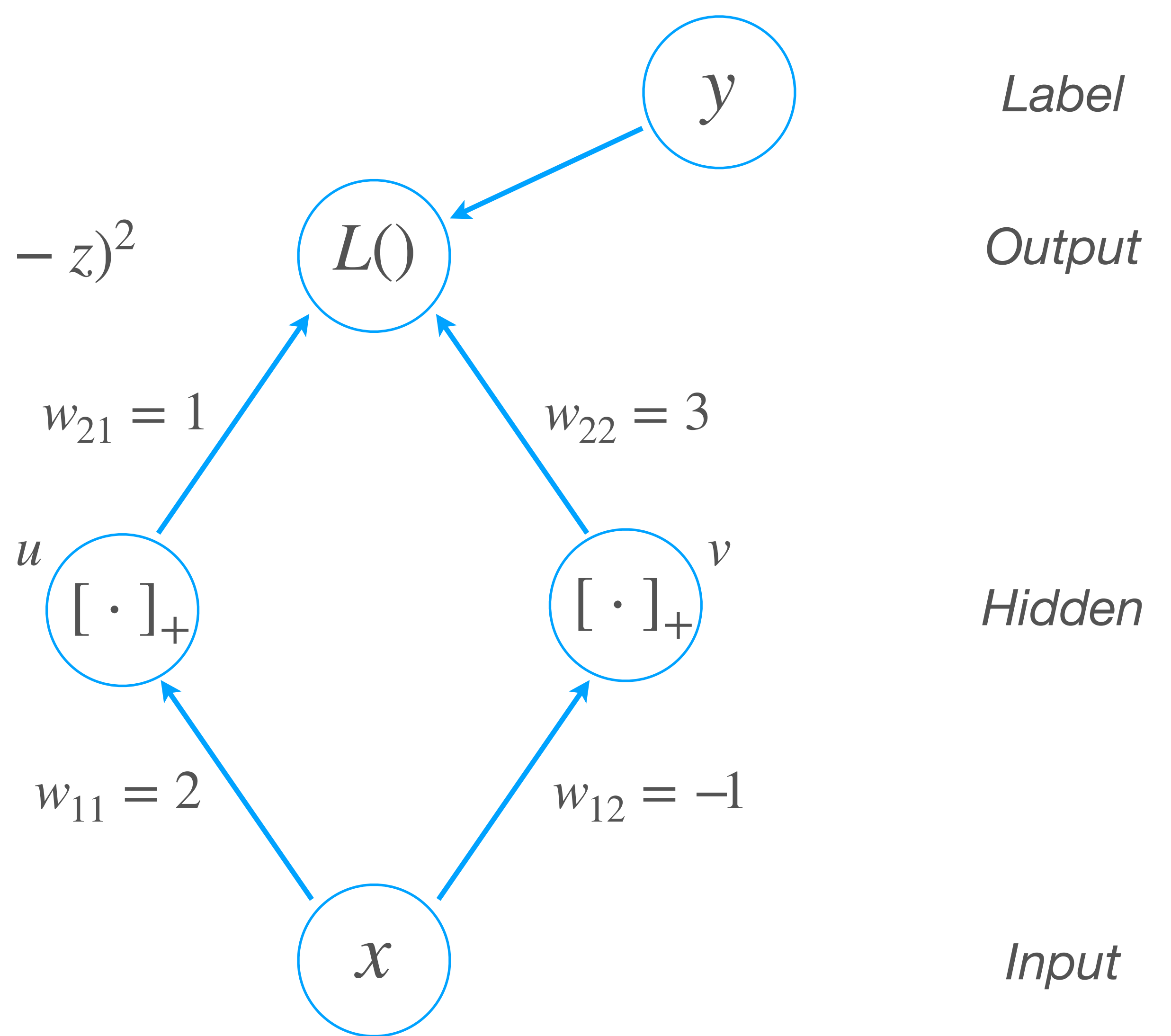
- Width, depth, work are only defined up to constant factors, since we can fiddle with the capability of the processors
 - ▶ convention for deep nets: a processor (a **neuron**) in one step computes linear function and fixed nonlinearity
 - ▶ a **layer** is a group of steps that can be performed in parallel

Example

a simple neural network predicting label y from input x under squared error

$$L(y, z) = \frac{1}{2}(y - z)^2$$

2 layers
width 2
work 3



each node computes a real number (an **activation**)
start at inputs, propagate toward outputs (**forward pass**)
by convention, weights label edges, but are part of the
computation of the incident node

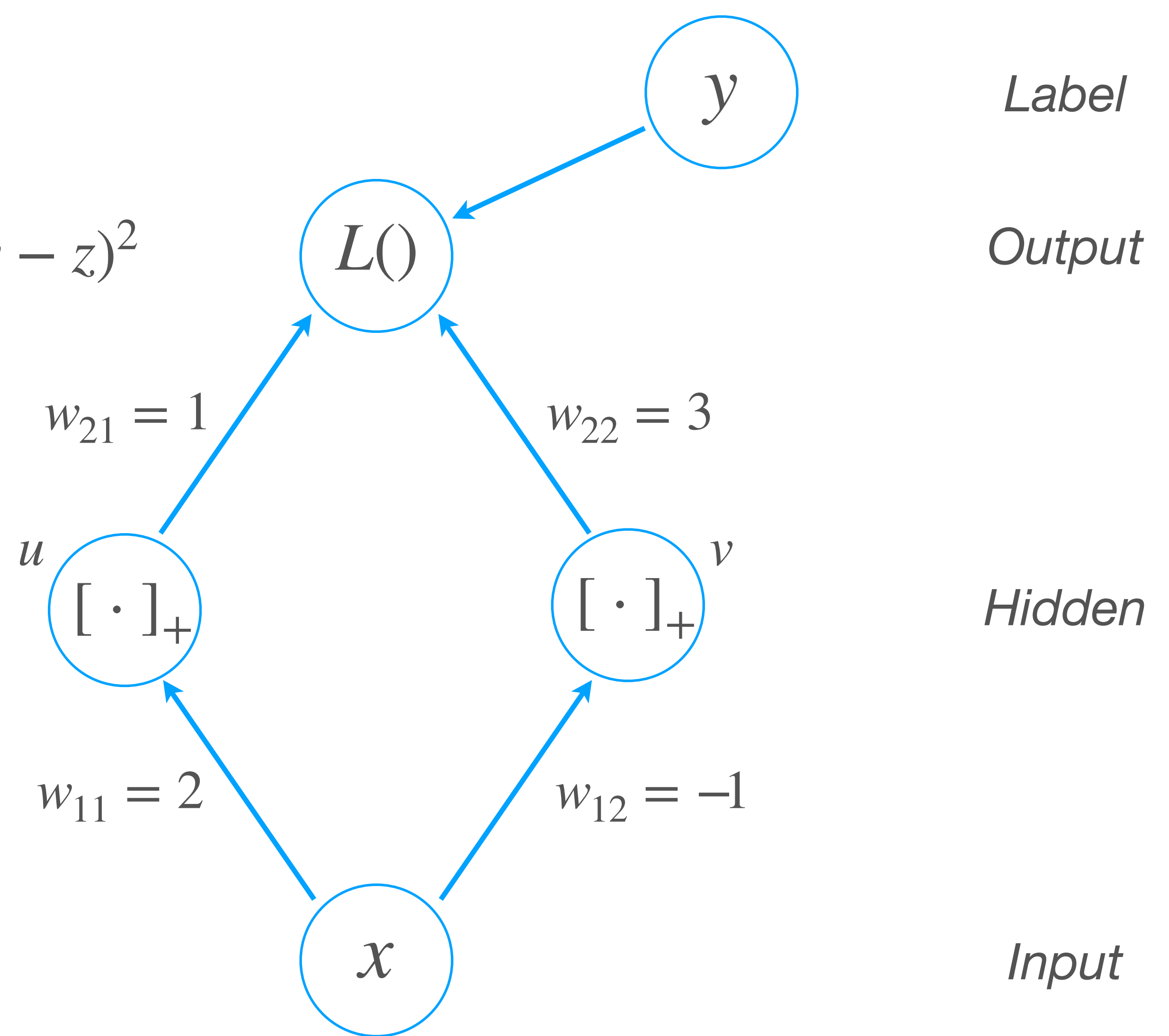
try $x = 1, y = 1$

Example

a simple neural
network predicting
label y from input x
under squared error

$$L(y, z) = \frac{1}{2}(y - z)^2$$

2 layers
width 2
work 3



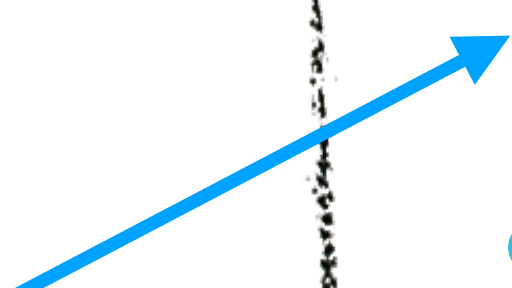
each node computes a real number (an **activation**)
start at inputs, propagate toward outputs (**forward pass**)
by convention, weights label edges, but are part of the
computation of the incident node

How to train your ~~dragon~~ deep net

- SGD!
- Draw examples (x, y) from training set (or minibatches)
- Run network forward from x to get a prediction
- Compare to y to calculate loss function
- Calculate gradient g : derivative of loss wrt network weights
- Update network weights by subtracting a multiple of g
 - ▶ perhaps w/ momentum, Adam, etc.

How to train your ~~dragon~~ deep net

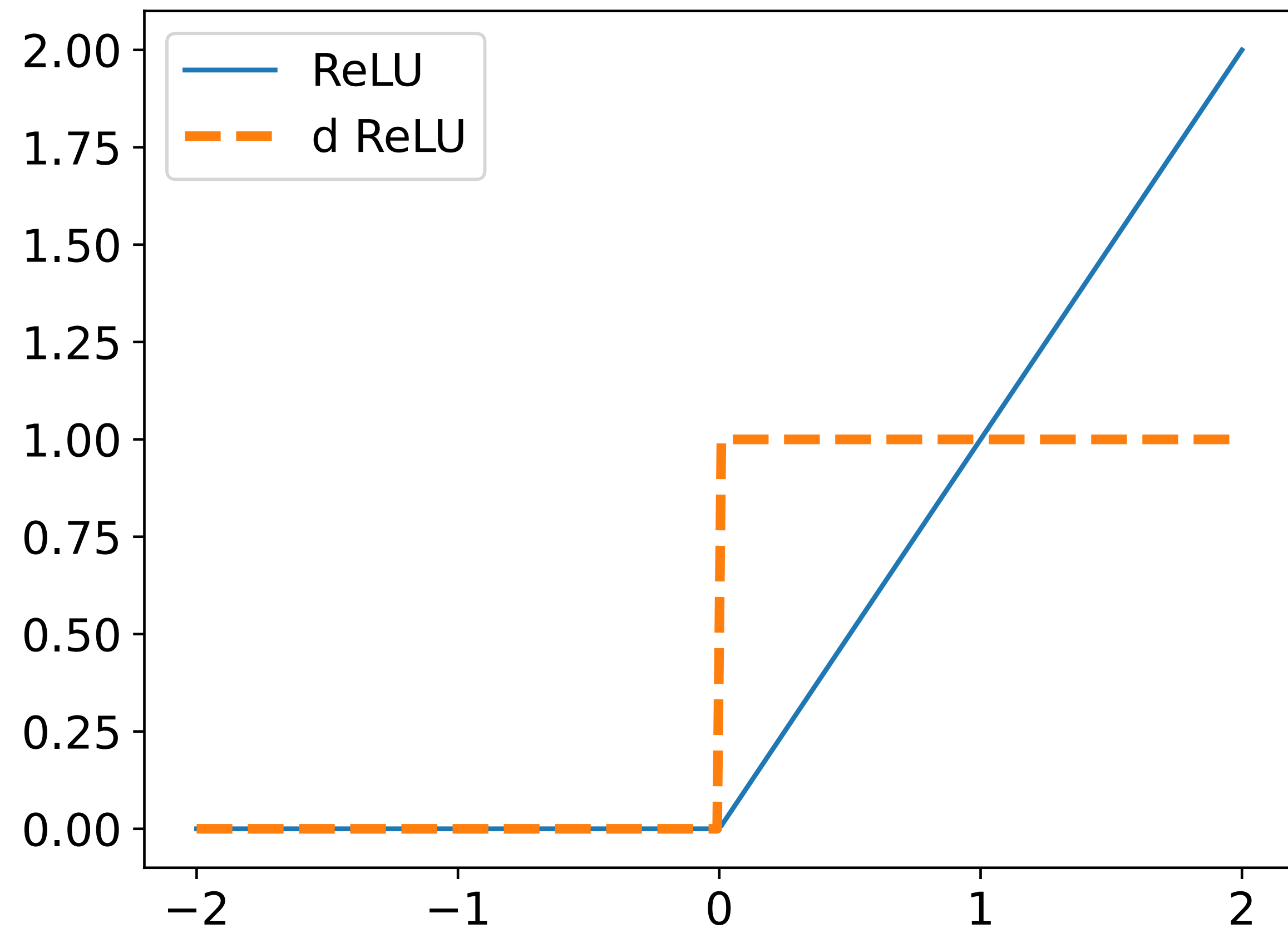
*how might we
do this?*



- SGD!
- Draw examples (x, y) from training set (or minibatches)
- Run network forward from x to get a prediction
- Compare to y to calculate loss function
- Calculate gradient g : derivative of loss wrt network weights
- Update network weights by subtracting a multiple of g
 - ▶ perhaps w/ momentum, Adam, etc.

Gradient of ReLU

$$[u]_+ = \begin{cases} u & \text{if } u \geq 0 \\ 0 & \text{if } u < 0 \end{cases}$$

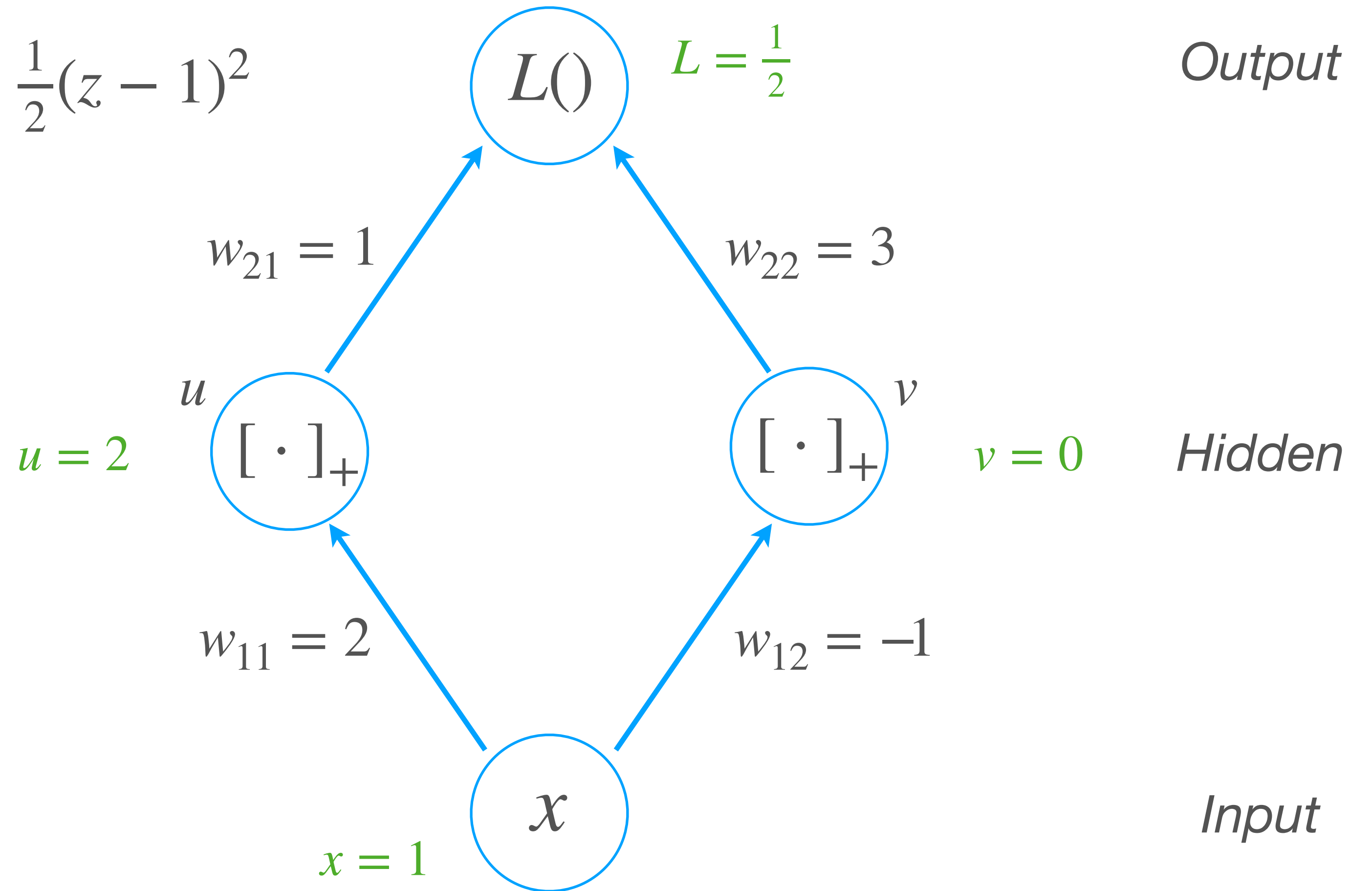


- Derivative of $[u]_+$: $S(u) = \begin{cases} 0 & \text{if } u < 0 \\ 1 & \text{if } u \geq 0 \end{cases}$

***Calculate
derivatives
wrt input x
and wrt
weights w_{ij}***

fix $y = 1$ for simplicity

$$L(z) = \frac{1}{2}(z - 1)^2$$



Professor provides

$$x = 1$$

$$x = 1 \quad w_{11} = 2$$

$$u = [w_{11}x]_+$$

$$x = 1 \quad w_{12} = -1$$

$$v = [w_{12}x]_+$$

$$u = 2 \qquad w_{21} = 1$$

$$v = 0 \qquad w_{22} = 3$$

$$L = \frac{1}{2}(w_{21}u + w_{22}v - 1)^2$$

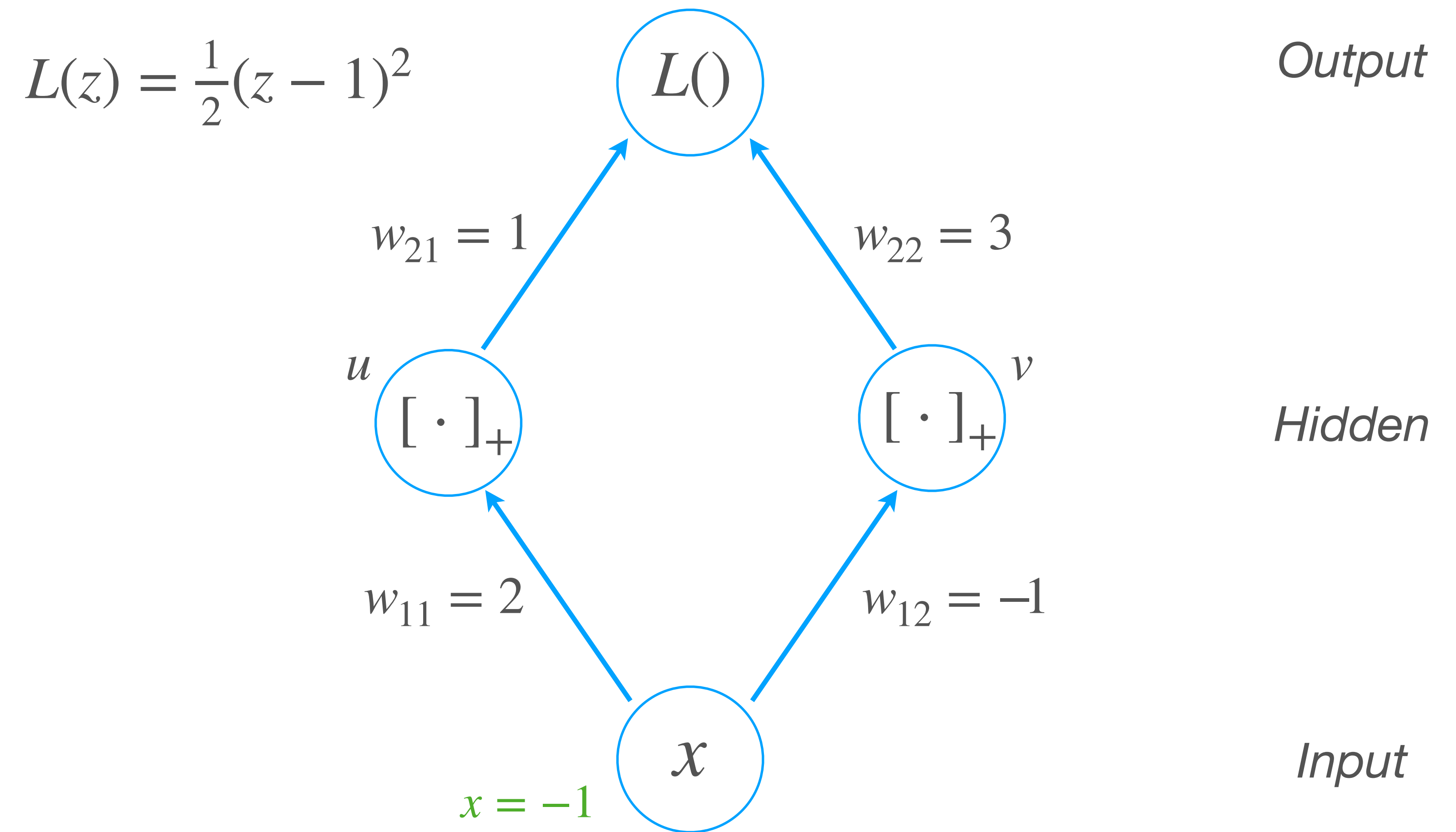
How did we get the derivative wrt x ?

- Loop: start above the top (output) node with $dL = dL$
- Visit nodes in backward order, from output to input (a **backward pass**)
 - ▶ so that we've looked at all successors of a node before we visit it, and know derivative of **overall** loss wrt **current node's** outputs
 - ▶ this is our loop invariant
- At each node, use chain rule to find derivative of loss wrt each **previous node's** outputs (this node's inputs) using known derivatives for current node

How did we get the derivative wrt w ?

- At each node, we already computed all of the information we need (activations from forward pass and intermediate gradients from backward pass)
 - ▶ weight gradients can then be computed locally
- Typical setup:
 - ▶ store activations during forward pass, accumulate weight gradients during backward pass
 - ▶ but it's possible to mix cacheing and recomputation to change tradeoff of memory vs. runtime

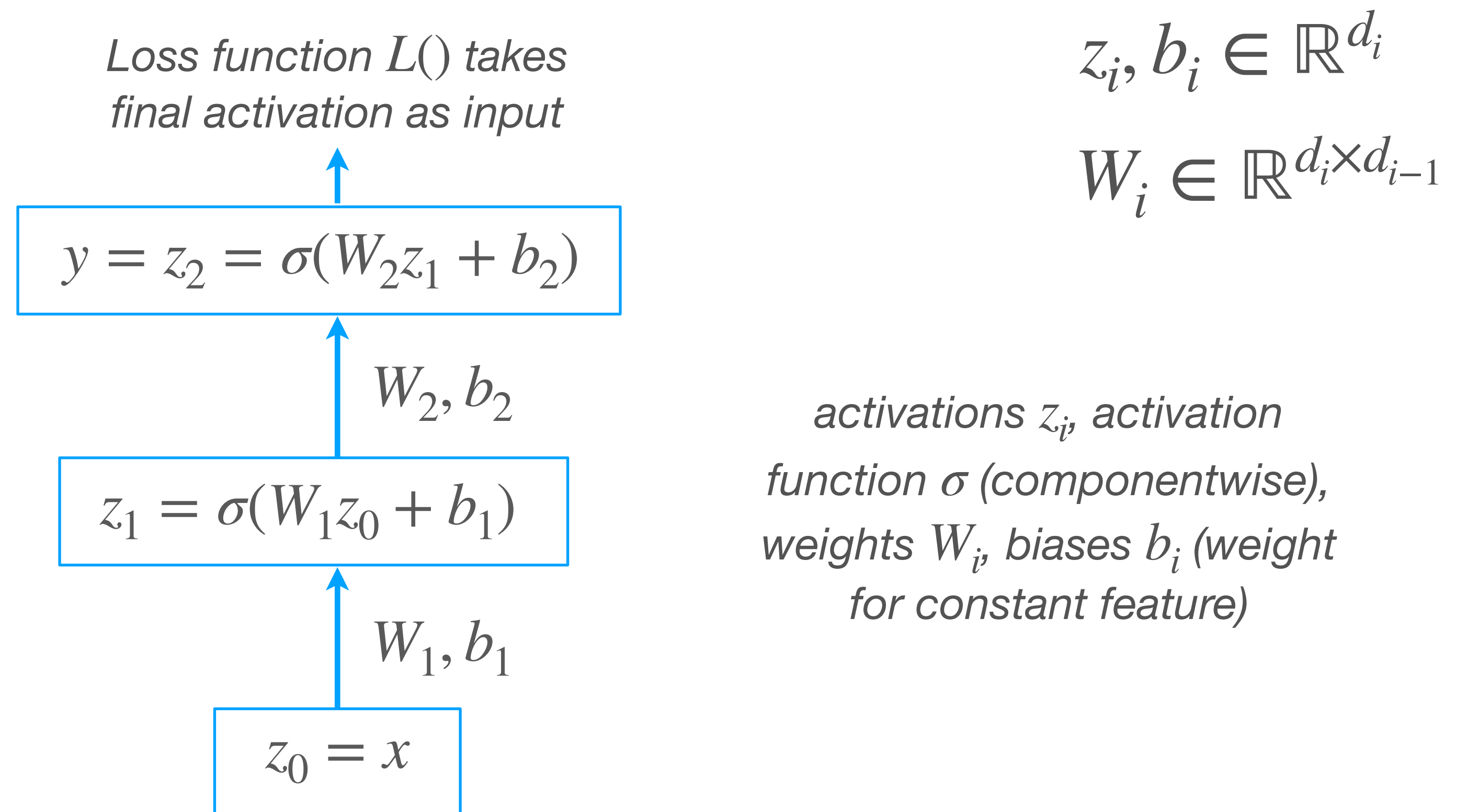
***Try another
example***



Backprop

- Forward and backward passes are ***automatable***
 - ▶ a few simple operations we do over and over
 - ▶ linear function, activation function, activation derivative, chain rule
- Resulting algorithm called ***backward propagation*** or ***backprop***
- Responsible for neural networks' initial rise to popularity

Backprop setup



- Suppose forward pass is: $z_i = \sigma(W_i z_{i-1} + b_i)$
 - ▶ that is, set $z_0 = x$, iterate $i = 1, 2, \dots, K$ computing **pre-activation** $u_i = W_i z_{i-1} + b_i$ and activation $z_i = \sigma(u_i)$
 - ▶ read off output $y = z_K$ and pass as input to $L()$

Backprop algorithm

- Goal: derivatives of loss L wrt weights W_i and biases b_i
 - ▶ as a byproduct, also wrt activation vectors z_i
- Loop invariant: we know $dL = \langle r_i, dz_i \rangle$
 - ▶ vector r_i is one of the activation derivatives
 - ▶ initialize at layer K w/ gradient of loss function (wrt its input, the final activation z_K)
- At each layer from K down to 1:
 - ▶ scale by derivative of activation f : define $v_i = f'(u_i) \circ r_i$
 - ▶ dL/db_i is just v_i
 - ▶ dL/dW_i is outer product $v_i z_{i-1}^\top$
- Update invariant: $r_{i-1} = W_i^\top v_i$

Fitting the model by SGD

- Sample an (x, y)
 - ▶ forward pass to compute $f_w(x)$ using weights w , saving intermediate values (hidden activations)
 - ▶ find loss, e.g. $L = \frac{1}{2}(y - f_w(x))^2$
 - ▶ backward pass to compute dL as a linear function of dw
- Take an optimizer step on w : e.g., step downhill using derivative (coefficient of dw)
- Or, for each (x, y) in minibatch
 - ▶ do above, accumulate derivatives (coefficients of dw)
 - ▶ step using sum of derivatives

Why depth?

- ML needs two things to work:
 - ▶ expressivity: model has to be able to represent (something close to) the thing we want to learn
 - ▶ simplicity: need a regularizer or complexity measure that rules out bad-yet-consistent hypotheses — depth, width, work, norm of weights, ...
- Tension:
 - ▶ expressivity: want to learn e.g. arbitrary Python code
 - ▶ simplicity: there are a *lot* of Python programs to search through, and we don't know how to rule out bad ones
- Depth seems to facilitate a tradeoff: simple families of deep programs contain useful functions at low complexity levels

Depth provably makes a big difference to expressivity

- For any k , compare programs that are
 - ▶ *shallow*: depth $\Theta(k)$ steps, or
 - ▶ *deep*: depth $\Theta(k^3)$ steps
- **Thm** [Telgarsky, 2016]: there are (many) functions that we can implement with *constant* width in deep programs but require *exponential* width $\Omega(2^k)$ even to approximate with shallow programs
 - ▶ step: can be quite general, e.g., piecewise polynomial fn of previous outputs

Why are classical models often shallow?

- Optimization
 - ▶ not obvious how to find good model in deep model class (need solutions to regularization, local minima, vanishing/exploding gradients, sharpness, ...)
- Generalization
 - ▶ deep model class typically has very high VC dimension / Rademacher complexity / anything else we can measure
 - ▶ need to co-design architecture and optimizer to get implicit regularization
- These used to be big problems
 - ▶ when I took 10-701:
“don’t try to use SGD to train a model with depth > 2 or 3”
- Spoiler: not any more

Fitting the model, autodiff version

- Use `Tensor` class for all arrays
 - ▶ keeps the “paper trail” of who depends on whom
- For each example in minibatch
 - ▶ run code that starts w/ (x, y) and ends w/ `loss += ...`
- Call `loss.backward()` to run autodiff
- Take an optimizer step (e.g., SGD) using accumulated derivative
- Clear derivative storage (e.g., SGD optimizer has `zero_grad()` method, as do most classes for models)

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Make it deeper

- Now that we have autodiff, no problem differentiating a 50-layer network!
- Let's try it:
 - ▶ $x, z_i, w_i, b_i \in \mathbb{R}$
 - ▶ $z_0 = x$
 - ▶ $z_i = \sigma(w_i z_{i-1} + b_i),$
i=1..50
 - ▶ initialize
 $w_i, b_i \sim N(0, 0.1^2)$
- Forward pass:
 $z_{50} = 0.5167$
- Backward pass: $\rightarrow$

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Make it deeper

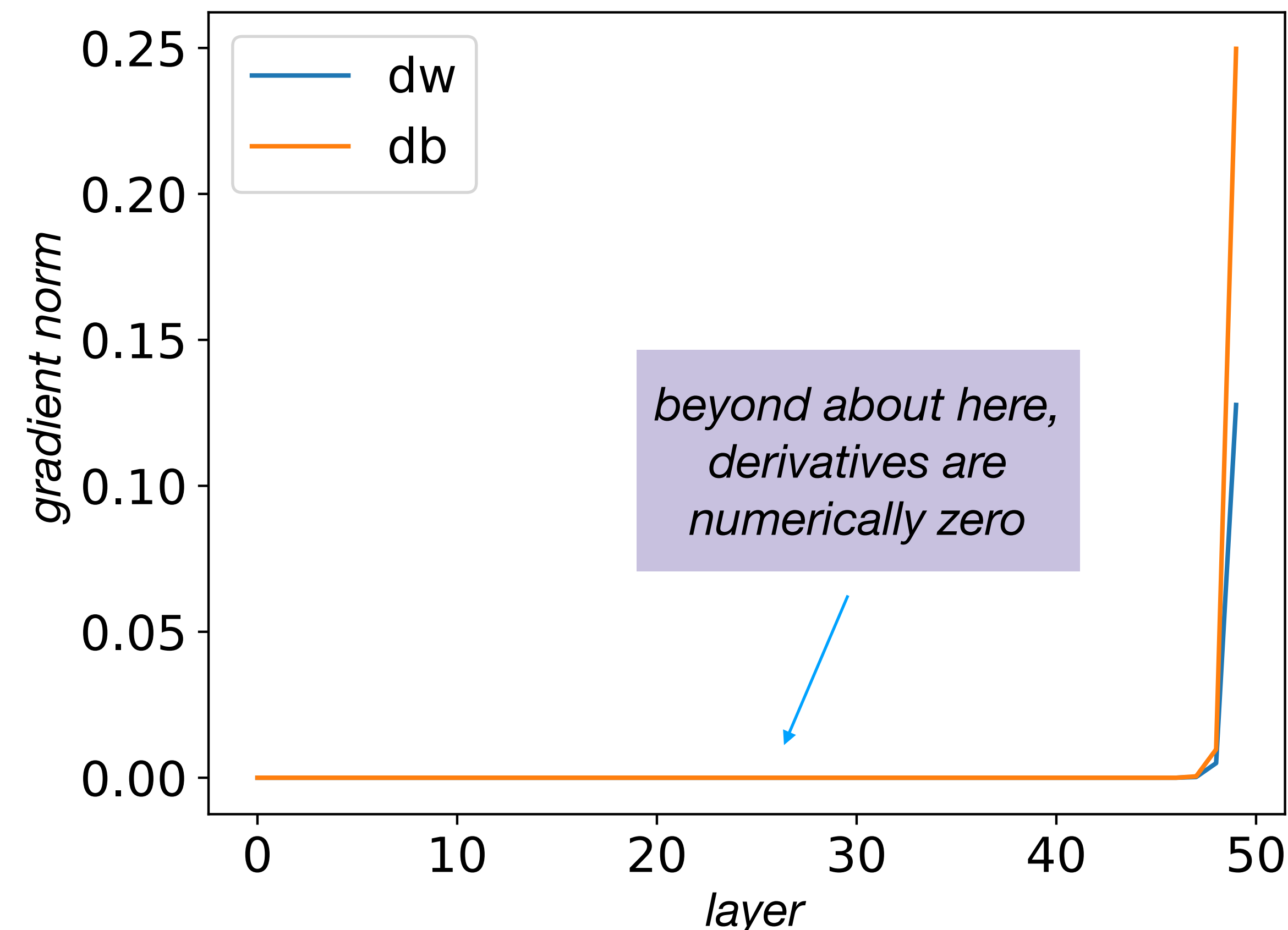
- Now that we have autodiff, no problem differentiating a 50-layer network!

- Let's try it:

- ▶ $x, z_i, w_i, b_i \in \mathbb{R}$
- ▶ $z_0 = x$
- ▶ $z_i = \sigma(w_i z_{i-1} + b_i), i=1..50$
- ▶ initialize $w_i, b_i \sim N(0, 0.1^2)$

- Forward pass:
 $z_{50} = 0.5167$

- Backward pass: →



Is this a problem?

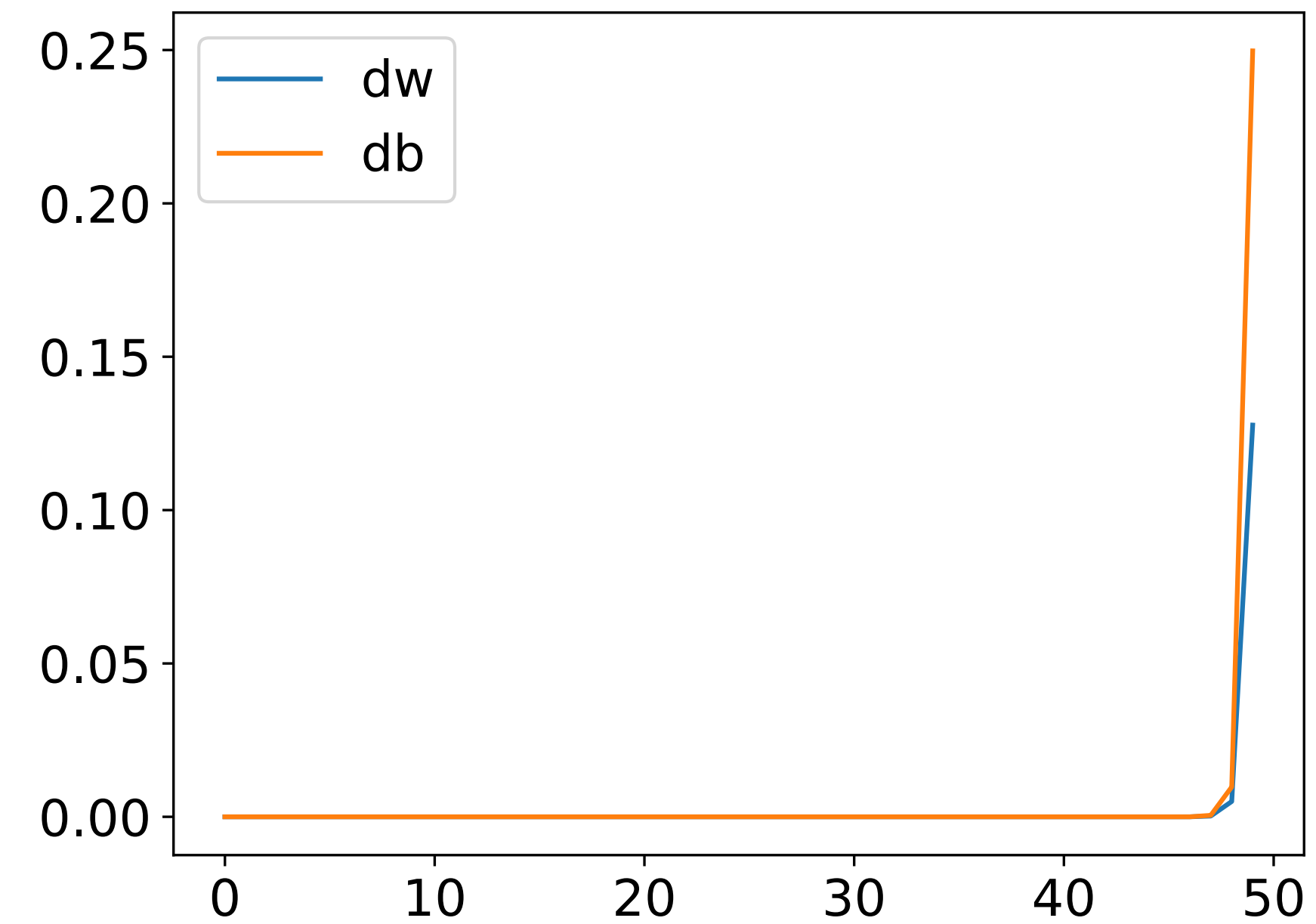
- In SGD: effectively no updates to w_i, b_i for $i \leq 45$, and truly zero updates for $i \leq 30$
 - ▶ so, we aren't really using a deep model: only last few layers are learnable
 - ▶ worse: in forward pass, $z_{30} \approx 0.5$ no matter what x is
 - ▶ $dz_{30} = 0 dx$ numerically
 - ▶ i.e., we ignore the input

Why does this happen?

- Run backprop:

$$\begin{aligned} dz_{50} &= \sigma'(w_{50}z_{49} + b_{50})w_{50}dz_{49} \\ &= \sigma'(w_{50}z_{49} + b_{50})w_{50}\sigma'(w_{49}z_{48} + b_{49})w_{49}dz_{49} \\ &= \dots \end{aligned}$$

- all σ' terms $\approx \frac{1}{4}$, all w_i terms ≈ 0.1 in magnitude
 - ▶ exponential decay: called *vanishing gradients*



Exploding gradients

- Can get a related problem if we initialize to *large* weights
- Keep multiplying gradient by $w_i \rightarrow$ exponential growth
- But growth can be self-limiting:
 - ▶ e.g., with sigmoid nonlinearity: $\sigma'(z) \approx 0$ when z large, numerically 0 if $|z| \geq 17$ or so
- So, paradoxically, explosion can also lead to (near-)zero gradients

Just right

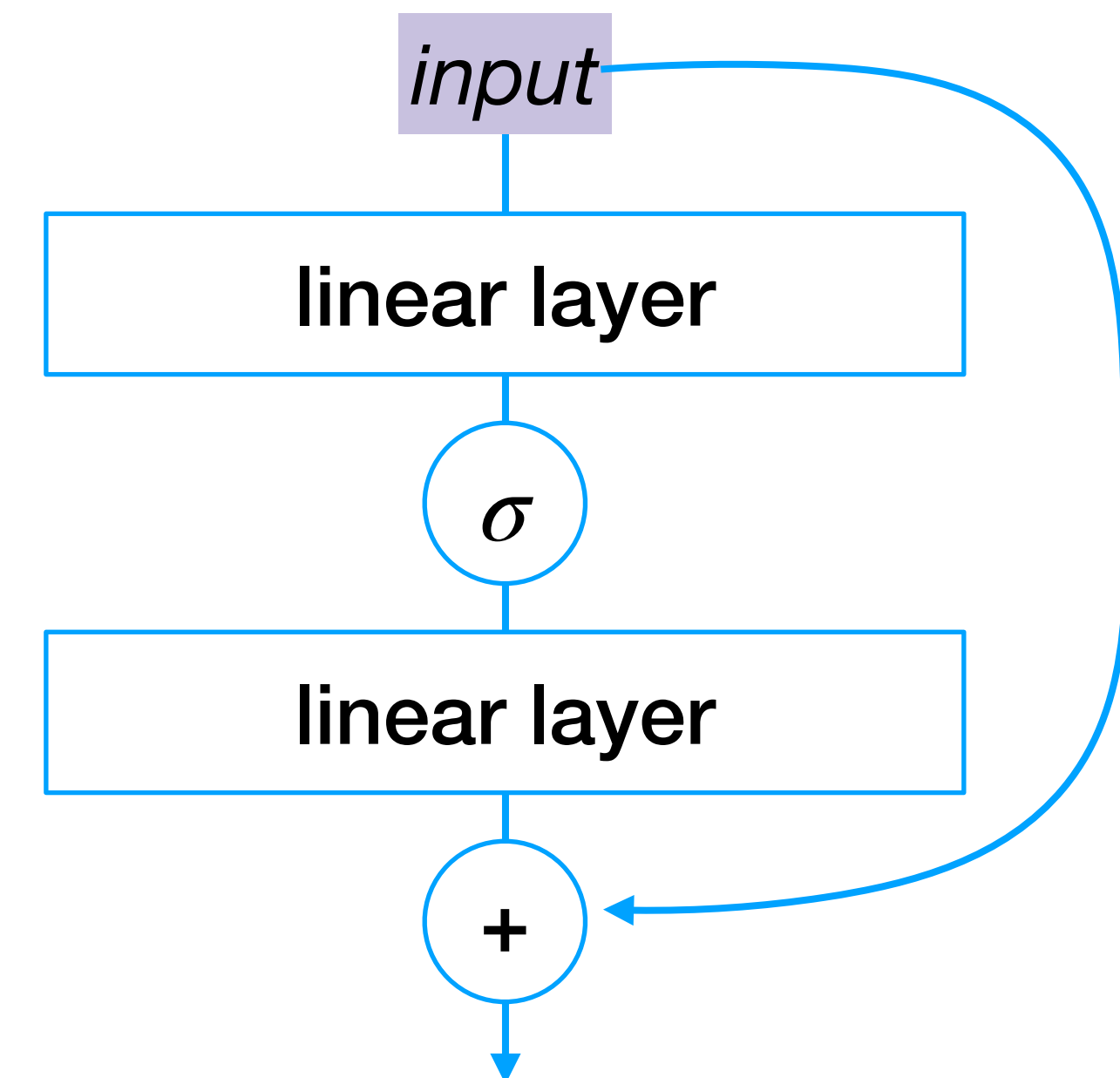
- What if we initialize to moderate-sized weights?
- Helps a lot
 - ▶ generally enough to train moderate-depth networks (say, 10 or so)
 - ▶ beyond that, it's hard to control for the contribution of σ'
- PyTorch does this by default
 - ▶ if we know approximate variance of vector z_t at layer t
 - ▶ and we choose variance V of elements of w_t
 - ▶ then we can calculate $\text{var}(z_t \cdot w_t)$
 - ▶ and multiply by an estimate of σ' to get variance of z_{t+1}
 - ▶ so adjusting V can keep gradients from growing or shrinking too much

Initialization matters

- Vanishing/exploding gradients
- Symmetry breaking
- $\exists$ results showing we can get a powerful model by appropriate random initialization, with training only adjusting the weights a tiny distance from starting values
- ...

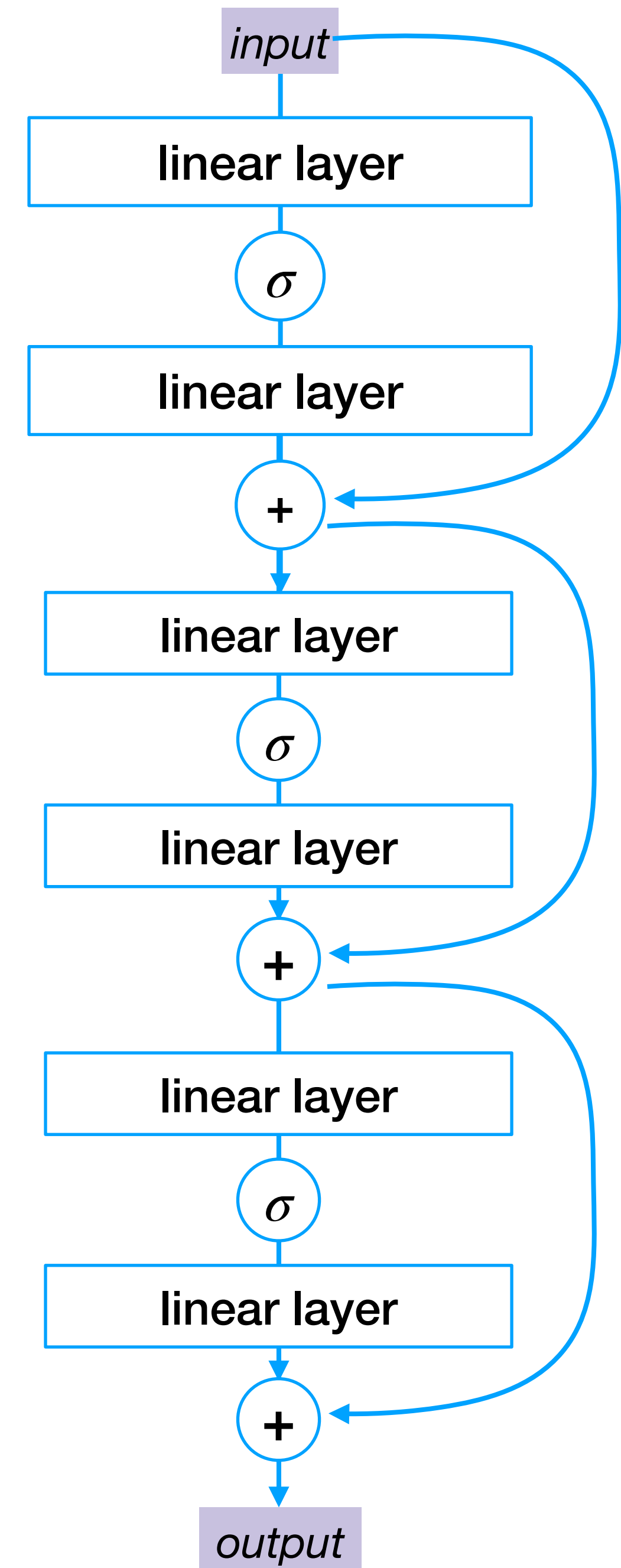
Better yet: residual connection

- Change the arrangement of layers:
 - ▶ $z_i = z_{i-1} + v_i \sigma(w_i z_{i-1} + c_i) + b_i$
 - ▶ can be any nonlinearity
 - ▶ weight matrices v_i, w_i
 - ▶ bias vectors b_i, c_i
- Sum node means we learn z_i as an *update* to z_{i-1}
- Second linear layer lets us learn sign of update, even if nonlinearity is always positive like $\sigma(z)$ or $[z]_+$



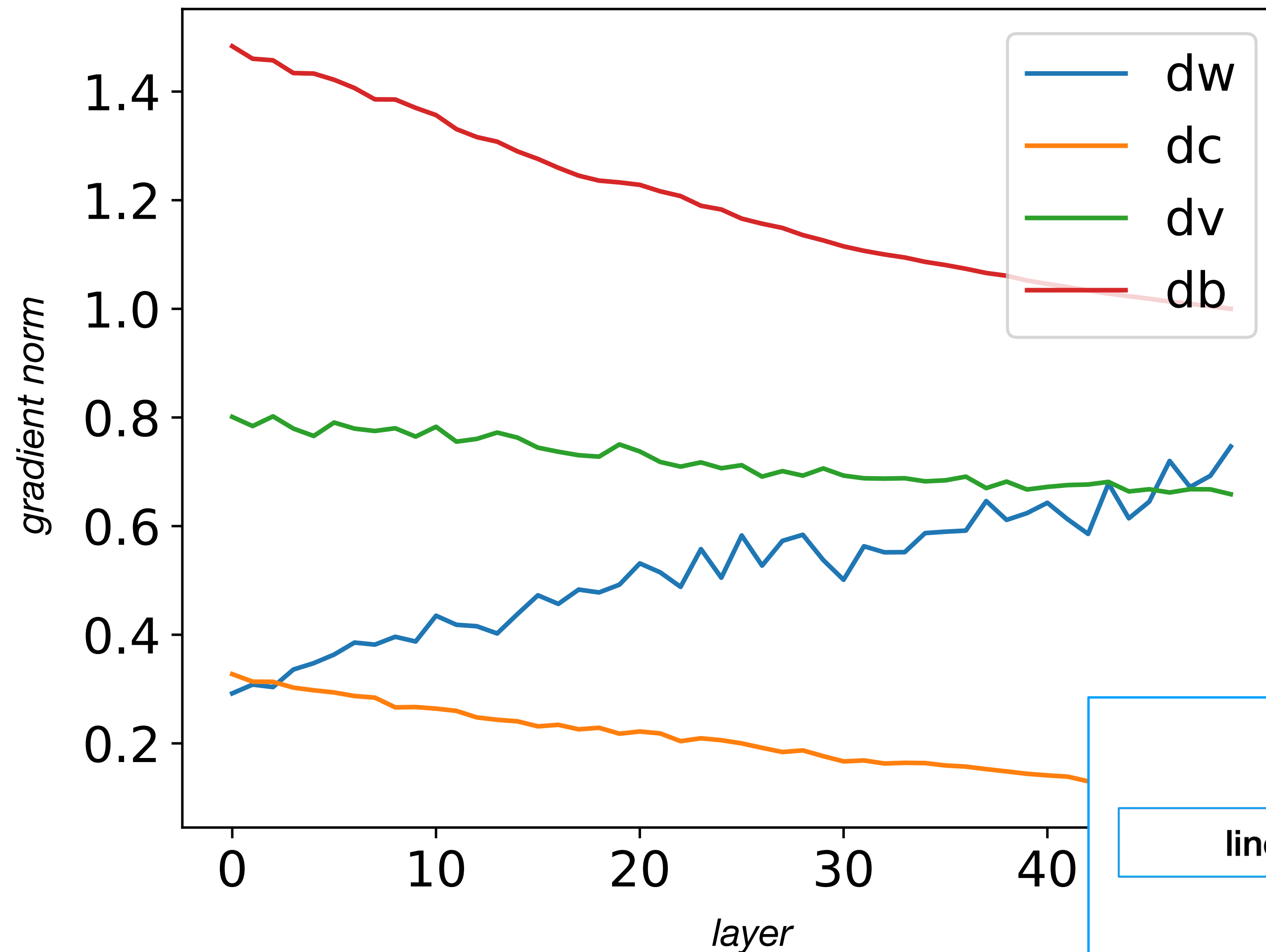
Residual networks

- Can stack residual blocks like any layer
 - ▶ appears in famous architectures like ResNet, Transformer
- Why does it prevent vanishing?
 - ▶ for each block, two paths for gradient to propagate back
 - ▶ one (the residual path) has factor 1

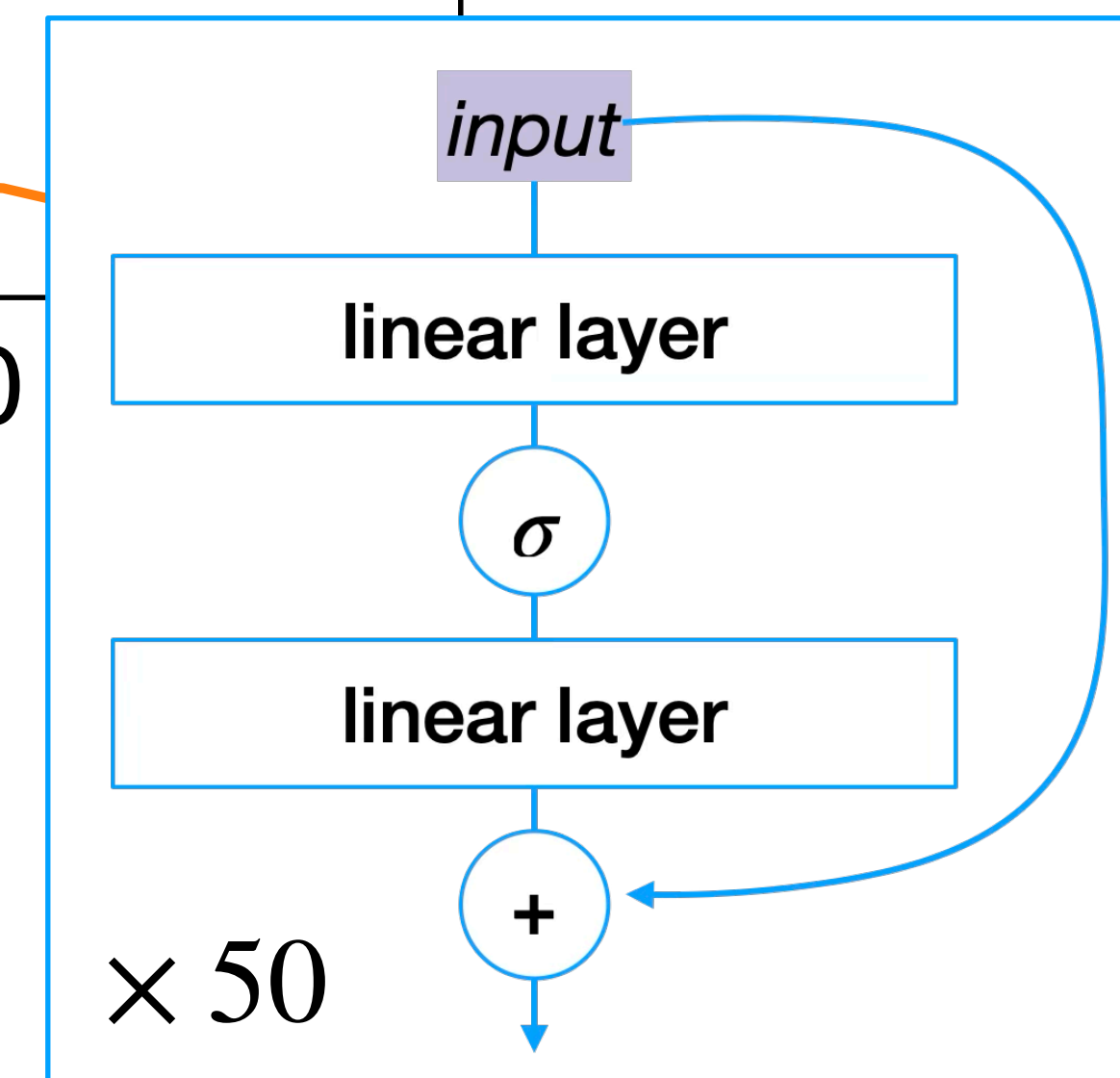


*This is an
architecture
diagram*

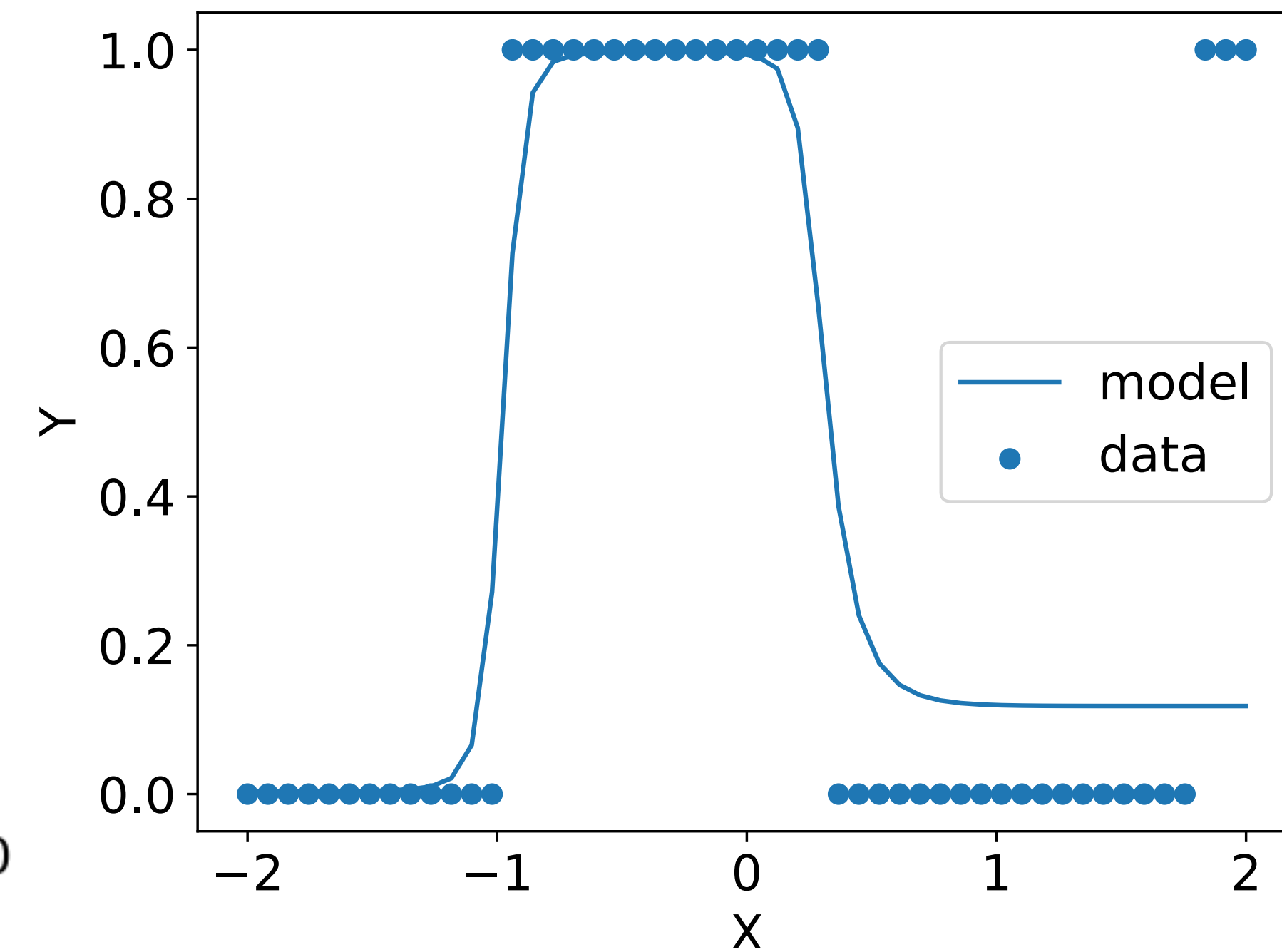
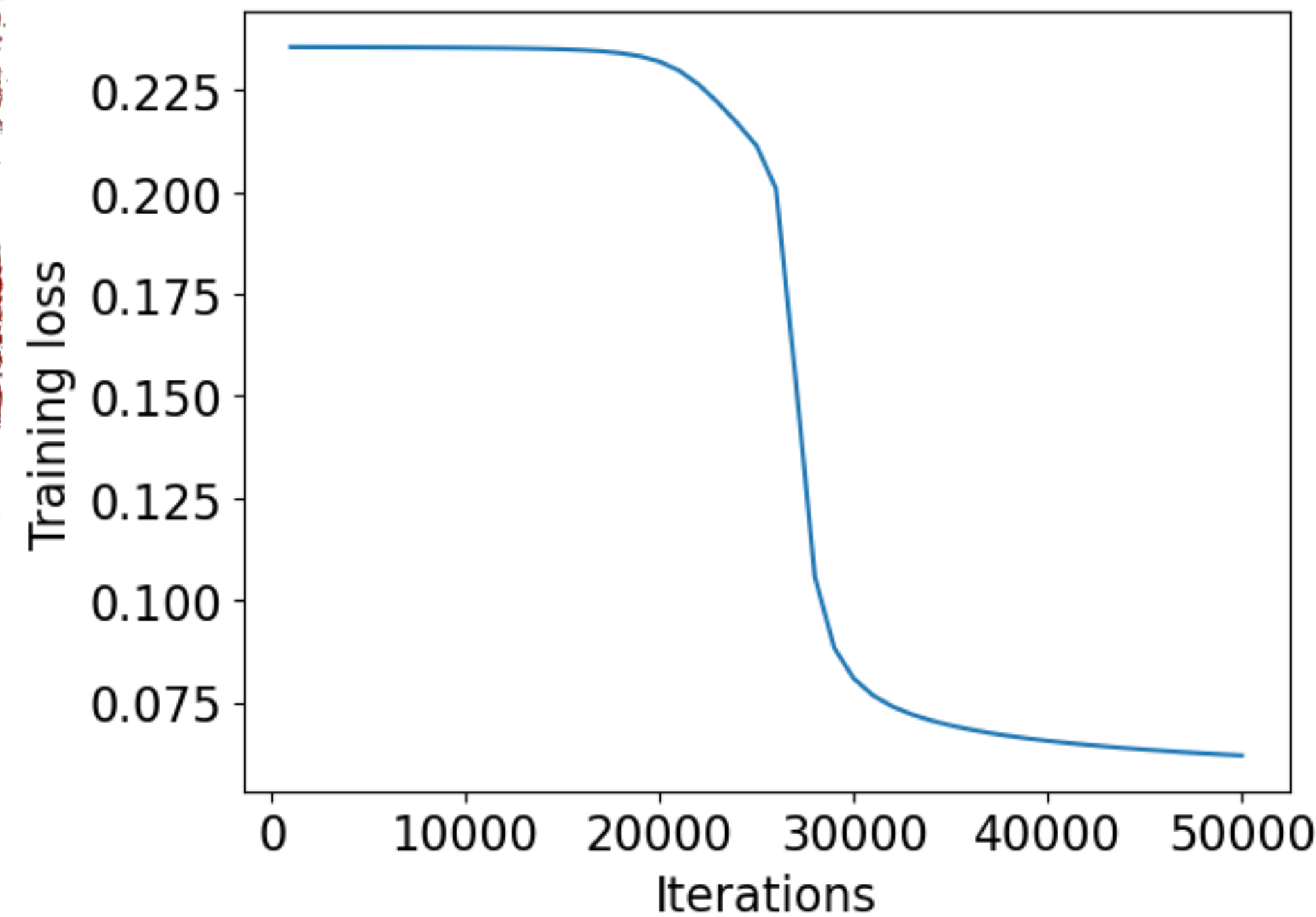
Gradient norm for residual blocks



● 50 stacked residual blocks

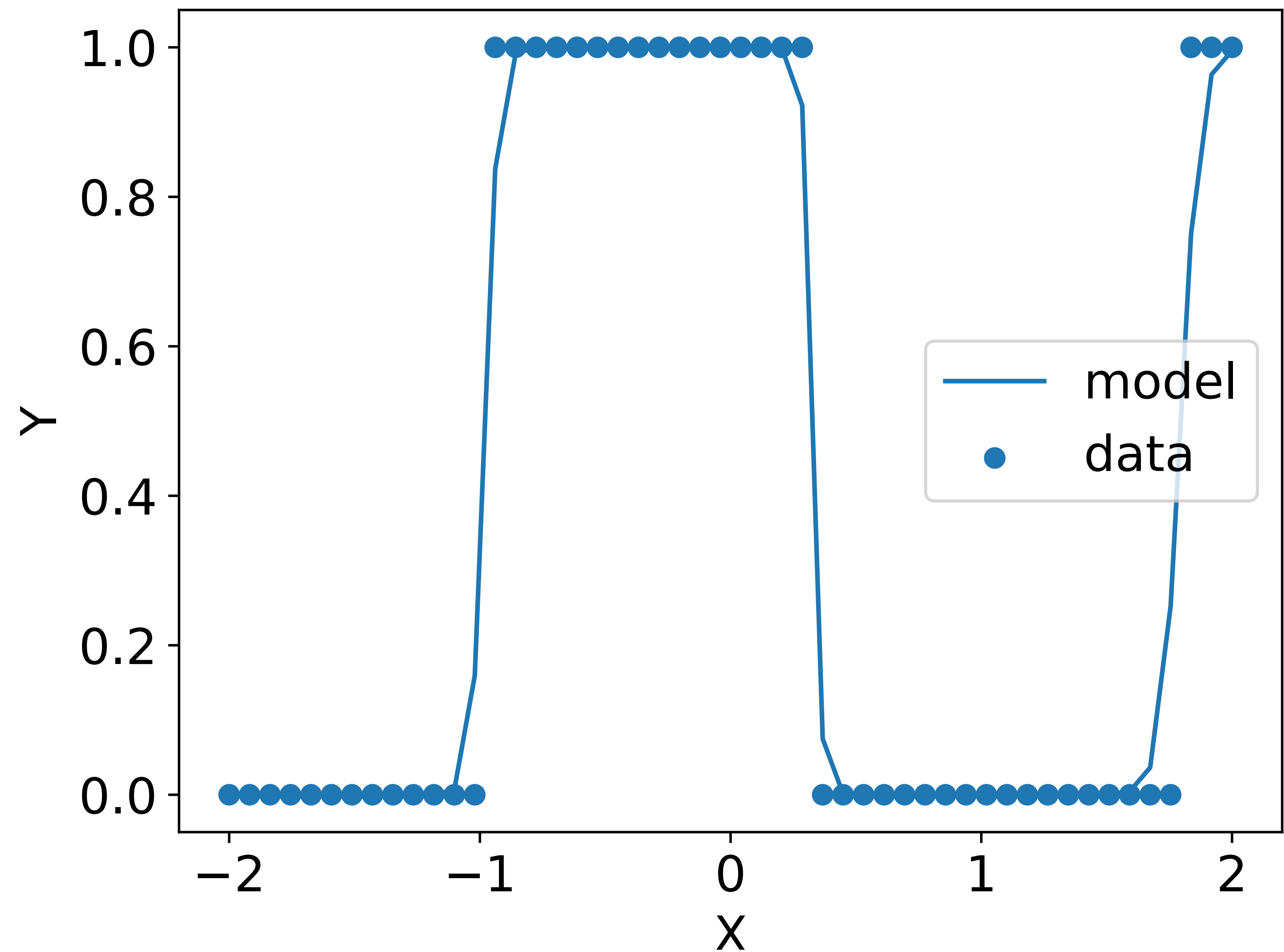


Bigger models seem to make GD/SGD work better



- Simple dataset of 50 points, 3-layer network
 - ▶ enough width to fit training data exactly
- GD, learning rate = 0.01, $\min \frac{1}{N} \sum_i (y_i - f_{W_{1:3}, b_{1:3}}(x_i))^2$
 - ▶ interestingly: GD doesn't find exact solution

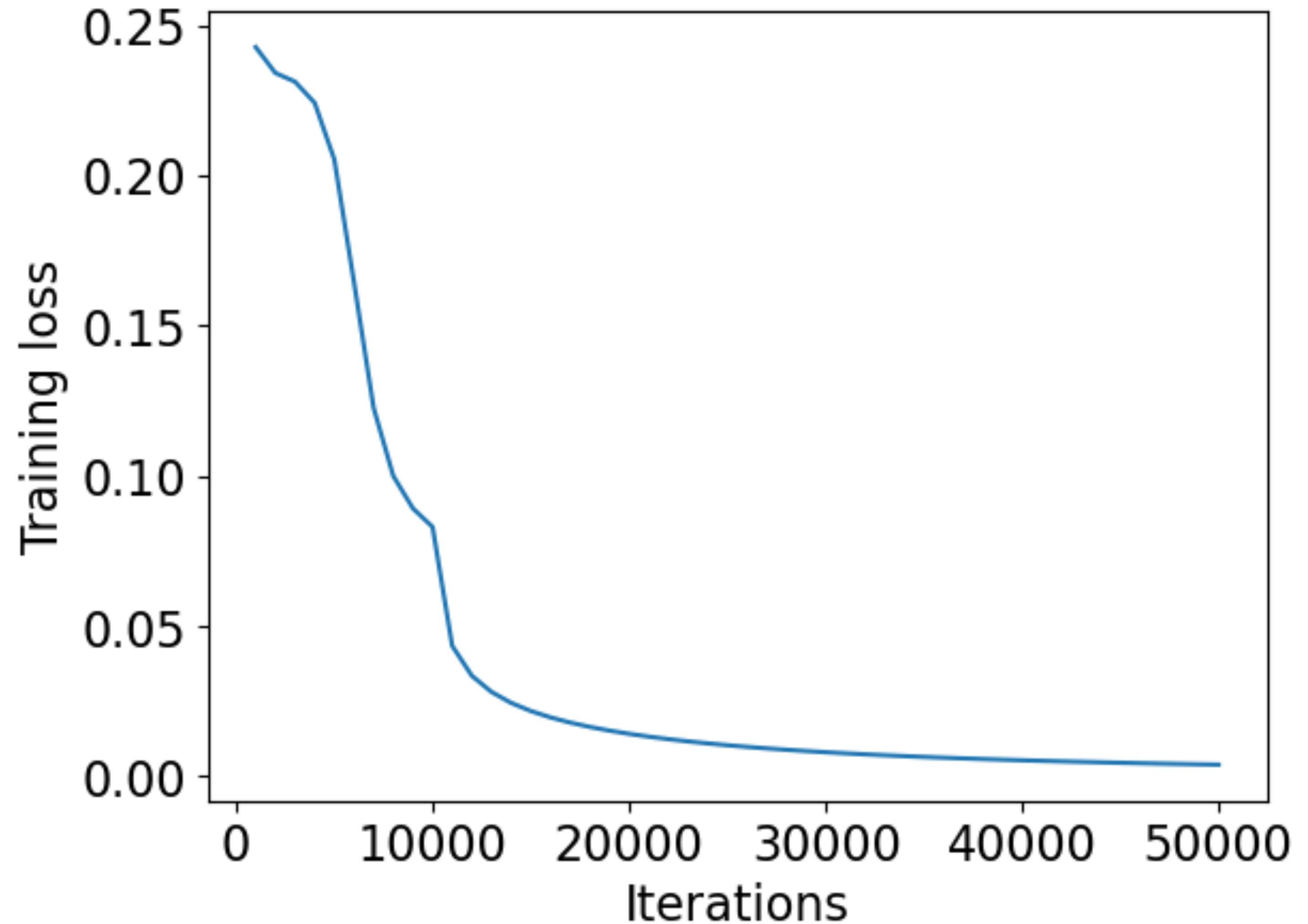
***Wider
network
finds a
better
optimum***



- Multiply width by 100 → perfect on training data

not obvious why

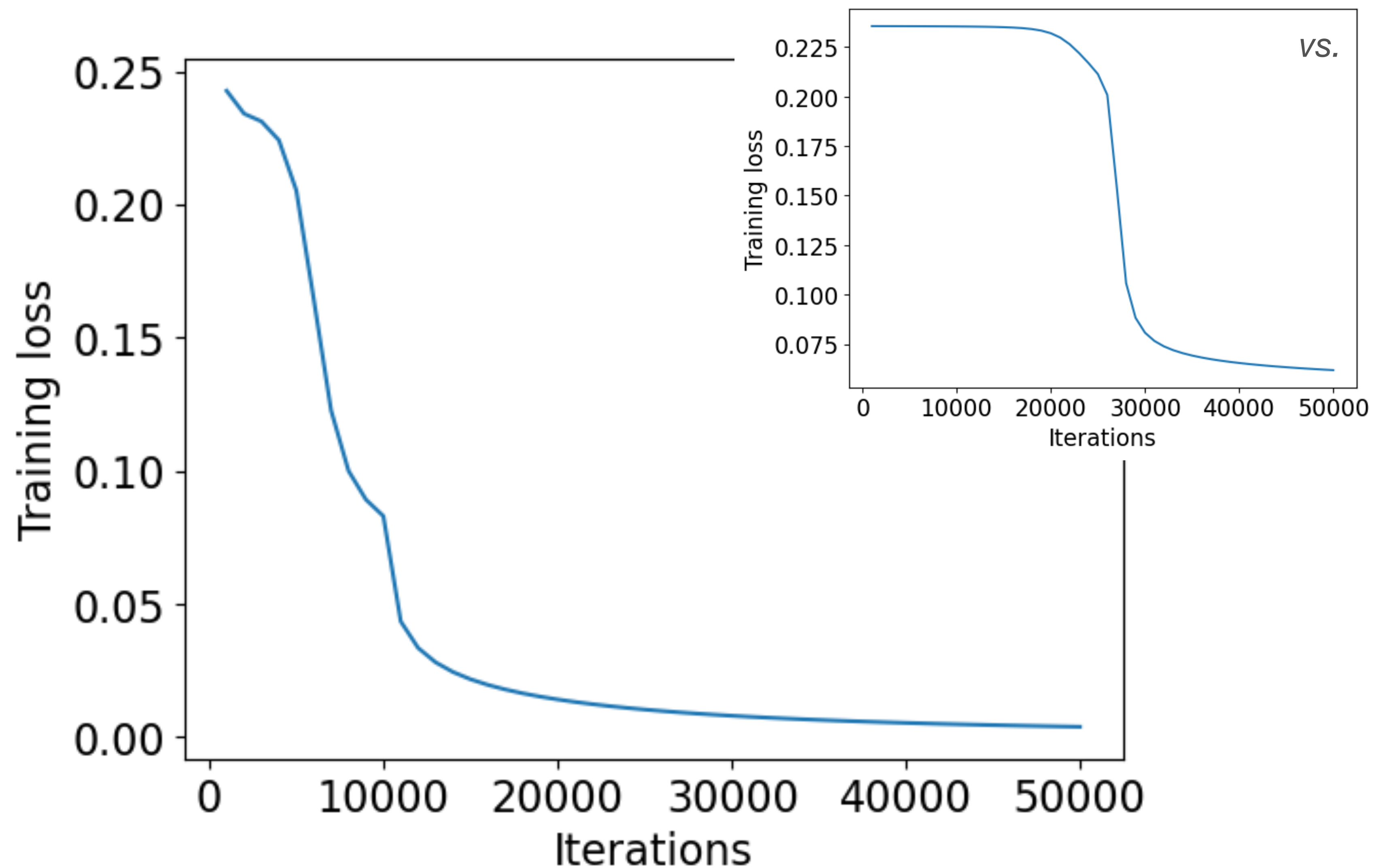
***Wider
optimizes
faster***



- Not just better classifier, but fewer epochs to train

not obvious why

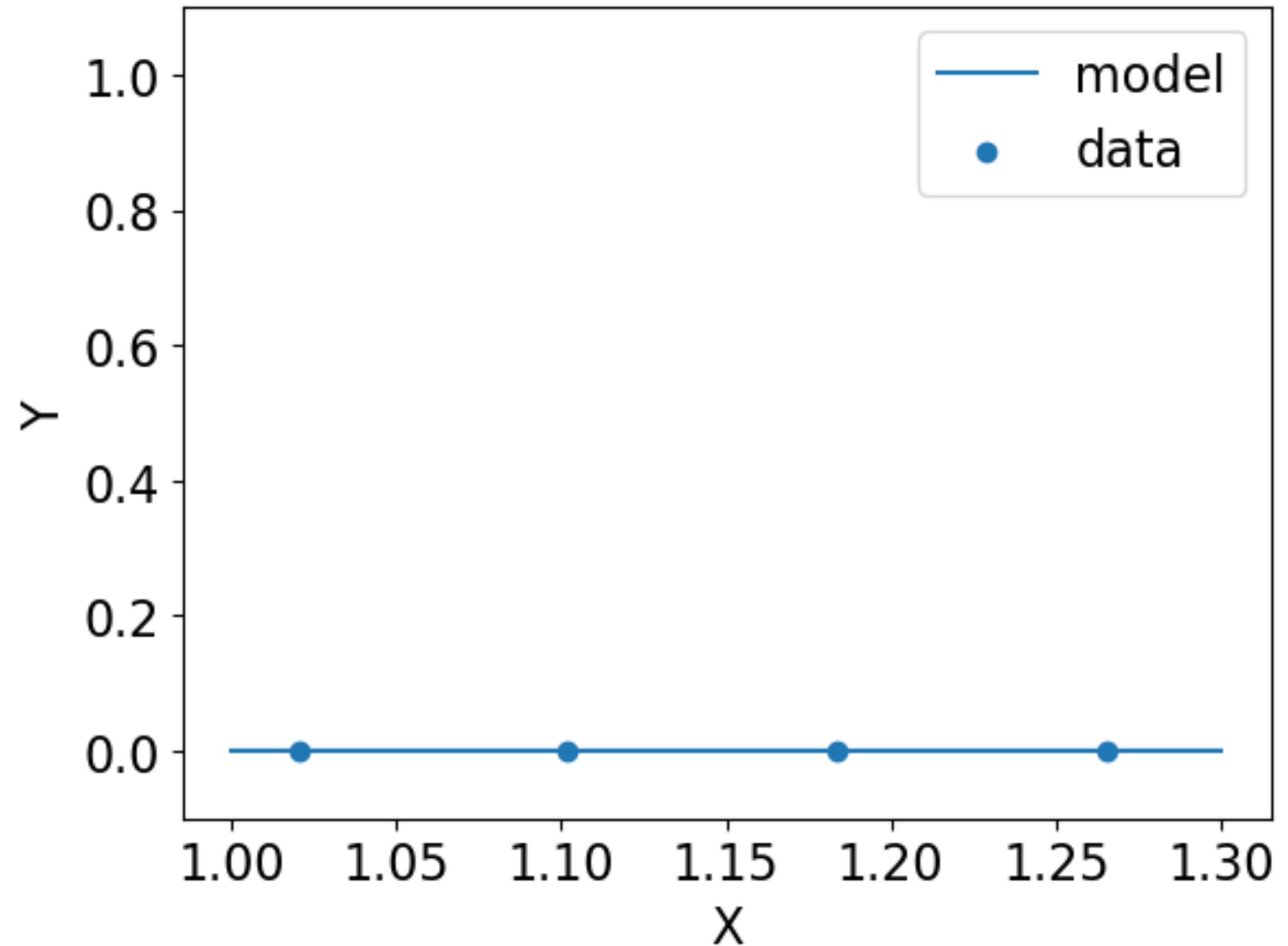
***Wider
optimizes
faster***



- Not just better classifier, but fewer epochs to train

not obvious why

Generalizes



- Not just better and faster, but generalizes

not obvious why